PUREMETAL FORGE

The ONNX Compiler Guide

Ahead-of-time ONNX compilation to readable BASIC source.

Generated 2026-10-19 · tools/build_help_pdf.py

Contents

What this compiler is

- Ahead of time, and only once
- What comes out is source. Not an executable.
- Why "source out" is the useful answer
- The four places the source can go
- What it will refuse
- Bind once, submit many

Building the tool from source

- You may not need to build it
- What you need, and what you do not
- The helper
- Or build the two entry points yourself
- What bin/ holds
- Check that it worked
- Why the runtime folder has to be there
- The private release hook, in one paragraph

Your first model

- Two models that cost nothing to get
- Generate the Windows source
- The same thing, in the window
- Build the consumer and run it
- What actually happened in those three lines
- Now try a packed weight
- Where to go from here

Reading a model

- Look before you compile
- The operator subset, in full
- Attributes: the value decides, not the presence
- The opset range it will admit

Fixed shapes and runtime dimensions

Pinning a dimension with --shape

Control flow and sequences

Random operators, and checking a model that has them

Errors are sentences

Precision: five choices, two mechanisms

What INT8 actually runs, and what it does not

Measured: the speech model at INT8

The earlier scheme on the Pi 4, and why it was replaced

What to do with your own model

What comes out

The set

The emitted source, from the top

The weight pack, and why the source checks it

The manifest

The runtime folder

Two APIs, and how to tell which you got

A model with random operators

Threads on Windows

The resident lifecycle, stated once

Watching a long run, and stopping it

The five targets

The target is the destination, not the desk

Windows

Pi 4 and UNO Q

The payload contract, in three lines of your Main

What the Pi 4 has actually done

Pico and Pico 2

One rule for all four bare-metal profiles

Speech, end to end

Speech is an application of the compiler, not a second compiler

The model is pinned, and the pin is enforced
The four assets, and their contracts
Preparing a voice
Preparing the pronunciation packs
Windows: the reading window
Pi 4 and UNO Q: the resident text adapter
A resident runner, in outline
What performance to expect today
Accuracy, stated the same way
Pico is not a destination for this

How it is proved

The oracle
Bit identity, where it can be claimed
What the published checks establish, precisely
The node-test score
Forms the node tests do not reach
Control flow and sequences: forms the node tests do not reach
Where the failure is, and why it is still recorded as a failure
The random operators: the same numbers on every target
InstanceNormalization: a reference that can be vectorised exactly
INT8: one definition, the same bits on every target
The operator set: one kernel source, the same bits everywhere
Threads on Windows: the same bits at every count
Where the gates live, and why not here
The limits, stated together
Before you trust your own model

Contributing

The shape of the tree
Which file should I change
The numerical contract
What the caller must have established

How a kernel is proved

Licensing and notices

Before you send a change

Where everything is

This book and the reference pages are different things

The map

Two things that are deliberately not here

How this book is rendered

Keeping it true

Running a model on the Raspberry Pi 4: the Kokoro sentence

What this chapter reproduces

What it measured

What you need

The inputs, pinned

Two builds of the compiler, and why

The compiler in the PureMetal Forge download

The model and the assets

The runner

The payload, byte for byte

On the board

Speaking something else

What this compiler is

It reads a model and writes source code. That sentence is the whole design, and most of the surprises come from taking it literally.

Ahead of time, and only once

Most ways of running an ONNX model work like this: a library loads the file, walks the graph, and for every node it looks up what to do and does it. The graph is still there while your program runs. Something has to interpret it.

This compiler does the work before your program exists. It reads the ONNX file, checks that every operator in it is one it understands, works out the shapes and the memory, and then writes out a source file containing the calls, in order, with nothing left to decide.

```
; from an emitted model, trimmed
Procedure PmModelStage1_0()
  DNode=0
  DBinary(2,1,1,0)
  If DError<>"" Or DCancel : ProcedureReturn : EndIf
  DRelease(1)
EndProcedure
```

That is a whole one-node model. `DBinary(2,1,1,0)` is "write tensor 2 from tensors 1 and 1, operation 0, which is add". There is no table of operator names in the finished program, no dispatch, and no graph. The schedule was decided here and got flattened into code.

> **Note.** This is why the generated program cannot run a model it was not

generated for. It is not a runner with a model loaded into it. It IS the model, expressed as a program.

What comes out is source. Not an executable.

The tool finishes when the source file is written. It does not then call a compiler to turn that source into a program, and it will not do it if you ask.

```
PASS: reusable source emitted to ...\twice.pb
Weights: ...\twice.pmw
Model requires one downstream build; requests use runtime dimensions.
```

"Requires one downstream build" is the tool telling you whose job the next step is. Yours. You open the emitted source in a compiler you already have and you build it, the same way you build anything else.

Four things it deliberately does not do, and none of them is an oversight:

- It does not invoke a downstream language compiler. There is no `--build`. - It does not install a toolchain, fetch a dependency, or download a model. - It does not upload firmware, open a serial port, or contact a board. - It does not link an inference engine into what it emits. The optional reference bridge in chapter 8 is a checking instrument for the desk, and nothing generated ever calls it.

> **Trap.** The one exception is easy to misread. `Build.pb` in the repository

root DOES run a compiler - the host language's own - but it builds THIS TOOL, not your model. It is developer infrastructure that happens to live in the same checkout. Chapter 2 uses it. Nothing in the model-generation path ever behaves that way, and if you are ever reading emitted code that seems to shell out to a compiler, something is wrong.

Why "source out" is the useful answer

It sounds like a limitation until you look at where the source is going.

A bare-metal program has no operating system underneath it. There is no loader, no file system unless you wrote one, no heap unless you provided one, and no agreement about where anything lives in memory except the one your application makes. A tool that tried to hand you a finished binary for that environment would have to guess your memory map, your startup code, and your build, and it would be wrong about all three.

Source sidesteps every one of those questions. The emitted model asks for two pointers and their lengths, and the application - which knows where its memory actually is - supplies them.

It also means you can read what you are about to run. The whole schedule is in one file in a language you can follow. When a number comes out wrong at node 1854, you can go and look at node 1854.

The four places the source can go

One tool, five target profiles, four kinds of destination:

```
> PureMetalOnnxCompilerCLI.exe --list-targets
windows Windows (native x64)
pi4     Raspberry Pi 4 (PureMetal AArch64)
unoq    Arduino UNO Q (PureMetal AArch64 UEFI)
pico    Raspberry Pi Pico (RP2040 Cortex-M0+)
pico2   Raspberry Pi Pico 2 (RP2350 Cortex-M33)
```

Windows emits host-language source for a native x64 application. This is the one you can build with nothing but the compiler you already used to build this tool, so it is where the quick start goes and where you should establish what the right answer looks like.

Pi 4 and **UNO Q** emit PureMetal BASIC for 64-bit Arm, with hand written Advanced SIMD kernels underneath the ordinary tensor calls. These are the profiles the speech work in chapter 7 runs on. They are built with the PureMetal Forge toolchain, which is a separate product and is not in this repository.

Pico and **Pico 2** emit PureMetal BASIC for the two microcontroller parts. They are for small models. Chapter 6 gives the numbers, and they are small numbers: the largest model a Pico will take has to leave room for your actual firmware.

> **Note.** The generator itself runs on Windows x64 in every case. The target

you select changes what it writes, not what it runs on. Choosing `pico2` on a desktop machine is normal and expected; it does not mean a Pico 2 is attached, and nothing about generation touches one.

What it will refuse

This is a compiler for a checked subset of ONNX, and the checking is the point. An operator it has not implemented is refused by name and by node:

```
> PureMetalOnnxCompilerCLI.exe --compile einsum.onnx --output out\e --target windows
ONNX COMPILER ERROR: node 0 uses unsupported operator Einsum
```

It also refuses opset versions outside the range it has actually audited, element types it does not carry, shapes it cannot justify, and model memory beyond a target's budget. Chapter 4 walks through the refusals with the exact text of each.

> **Trap.** A refusal is the good outcome. The failure mode this whole project

is built to avoid is a tool that accepts something it does not really support and emits code that runs, produces numbers, and is quietly wrong. That costs somebody a week. An error costs them ten minutes. If you find a graph that is accepted and produces the wrong answer, that is a defect worth more attention than any refusal.

Bind once, submit many

The last piece of the design shows up in every chapter after this one.

A generated model is not a function you call with a file and get an answer from. It is something you set up once and then feed. You give it memory and weights, it validates them and keeps them, and from then on you fill inputs, execute, read outputs, and go round again. The weights are checked and decoded exactly once, however many requests you make.

For the speech model in chapter 7 that distinction is the difference between a usable design and an impossible one: binding 324 MB of weights is not something you want to do per sentence.

```
PmModelInitialize(weightPath)   ; once
  PmModelResetRequest()         ; per request
  ... fill inputs ...
  PmModelExecute()
  ... read outputs ...
PmModelClose()                  ; once, at the end
```

Chapter 5 describes both emitted APIs properly, including the fixed-shape one, which takes its memory from you rather than allocating any.

Building the tool from source

Two executables, one helper, and nothing to install first.

You may not need to build it

The easiest way to get this compiler is the PureMetal Forge download from puremetalforge.ajtaji.com. It already includes the compiler, built and ready to run, in `PureMetalForge\tools\onnx\`:

```
PureMetalForge\tools\onnx\
  PureMetalOnnxCompiler.exe       the window
  PureMetalOnnxCompilerCLI.exe    the command line
  NATIVE_COMPILER_GUIDE.txt       the user guide
  README.md
```

That is the same pair this chapter builds into `bin/`, and nothing else is needed to use it: no PureBasic. The two checks at the end of this chapter give the same output from the download's CLI. The same download is the PureMetal toolchain that builds the Pi 4, UNO Q, Pico and Pico 2 output.

Building from source is for people changing the compiler, and for anyone who needs work committed to this repository after their download was built.

> **Trap.** A download carries the compiler as it was when that release was

built, and this repository moves faster than releases. The 2026-09-14 release's executables were built on 2026-09-13. They run the chapter 3 walkthrough, but they cannot build chapter 11's Kokoro sentence: that runner calls `PmOnnxAnvilEnter`, which the model source gained later, and the download's supplementary pronunciation pack is the older one. The 2026-09-17 release's executables are commit `658566d`: they build the sentence and every asset, but its payload comes from later kernels than the one that ran. Chapter 11 shows exactly what differs.

What you need, and what you do not

You need the host language's Windows **x64** compiler. That is the whole list, and its C back end (`pbcompilerc.exe`) builds everything. The tool has been built and verified with version 6.41 and that back end.

You do not need Python, an ONNX package, an ONNX Runtime library, a model file, or a PureMetal Forge installation. None of those are required to build the generator, and none of them are required to run the arithmetic walkthrough in chapter 3. The complete build-source closure is in this checkout.

> **Trap.** Build **x64**. The 32-bit compiler will produce an executable that

looks fine and then behaves badly on real models, because the tool works in 64-bit offsets throughout. `Build.pb` refuses to compile at all under a 32-bit compiler, and that refusal is deliberate - it is cheaper than the mystery.

Linux and macOS hosts have not been validated. The window and the emitted Windows execution sources use Windows APIs directly. That is a statement of what has been tested, not a claim that the parser cares.

The helper

Open `Build.pb` in the 64-bit IDE and run it. It finds the compiler that belongs to that installation, creates `bin/` if it is missing, and builds both entry points in order:

```
Building bin\PureMetalOnnxCompilerCLI.exe
Building bin\PureMetalOnnxCompiler.exe
Build complete. Open bin\PureMetalOnnxCompiler.exe.
```

If either build fails, the helper prints the compiler's own output, says `BUILD FAILED`, names the source and the exit code, and stops. It does not go on to the second build and it does not leave you with one new executable and one stale one.

Or build the two entry points yourself

The helper is a convenience, not a secret. It runs exactly these two commands, and so can you:

```
$pb = 'C:\Path\To\Compilers\pbcompilerc.exe'
New-Item -ItemType Directory -Force bin | Out-Null
& $pb src/PureMetalOnnxCompiler.pb   /CONSOLE /THREAD /OPTIMIZER            /OUTPUT
bin/PureMetalOnnxCompilerCLI.exe
if ($LASTEXITCODE -ne 0) { throw 'CLI build failed' }
& $pb src/PureMetalOnnxCompilerUI.pb          /THREAD /OPTIMIZER /DPIAWARE  /OUTPUT
bin/PureMetalOnnxCompiler.exe
if ($LASTEXITCODE -ne 0) { throw 'UI build failed' }
```

In the IDE, the same thing: open each source, select the C back end, set the listed options, and use Create Executable with the matching output name.

The options are not decorative. `/CONSOLE` is what makes the CLI a console program instead of one that opens a window. `/THREAD` is required because the window runs generation on a worker thread while staying responsive. `/OPTIMIZER` is what the verified builds use. `/DPIAWARE` keeps the window sharp on a scaled display.

What bin/ holds

Two executables, and they are a pair.

`PureMetalOnnxCompilerCLI.exe` is the engine. Everything the tool can do, it does here: inspecting, checking, generating, packing assets.

`PureMetalOnnxCompiler.exe` is the window. You pick a model, an output prefix, a target and a precision, and press the button. It then launches the CLI sitting beside it and shows you what the CLI said.

> **Trap.** Keep the two together. The window locates the CLI as its sibling in

the same directory. Copy one of them somewhere on its own and the window will have nothing to run. It is not hiding a second copy of the compiler inside itself - there is only one implementation, and the window is a client of it.

> **Note.** `bin/` is not tracked by this repository, and neither is anything

you generate into `output/`. The repository holds source and documentation. Your build is yours.

Check that it worked

Two commands, and neither needs a model:

```
> .\bin\PureMetalOnnxCompilerCLI.exe --self-test
PASS: bounded protobuf wire reader, native ONNX model inventory and PMG2P dictionary core

> .\bin\PureMetalOnnxCompilerCLI.exe --list-targets
windows Windows (native x64)
pi4     Raspberry Pi 4 (PureMetal AArch64)
unoq    Arduino UNO Q (PureMetal AArch64 UEFI)
pico    Raspberry Pi Pico (RP2040 Cortex-M0+)
pico2   Raspberry Pi Pico 2 (RP2350 Cortex-M33)
```

`--self-test` exercises the bounded protobuf reader, the model inventory and the pronunciation-pack core against built-in cases. It is a sanity check on the build you just made, not a proof that every graph compiles. Chapter 8 is about what the real proofs are and where they live.

`--help` prints the complete command list. It is the authority on flag spelling, and this book quotes it rather than paraphrasing it.

Why the runtime folder has to be there

`runtime/` looks like it belongs to generated programs, and it does - but the generator needs it at ITS build time, not just at generation time.

The emitters embed the runtime files into the tool with an include-the-file-as-bytes instruction, and the reduced-weight decoder and the shared speech sources are included as source outright. So a checkout missing `runtime/` does not produce a tool that generates broken models. It produces a tool that will not build.

> **Trap.** Two consequences catch people, and they are the same fact twice.

Change a runtime file and you must rebuild the generator before the change can reach anything. And **an already-generated application does not update itself**: its runtime was copied out beside it at generation time. To pick up a runtime change, regenerate the model's source closure and build the application again. Mixing an old generated runtime with a newly generated model source is how you get a mismatch nobody can see.

The private release hook, in one paragraph

If you are maintaining a product that ships these executables, an ignored `build.local.ini` can name a PowerShell script, and `Build.pb` will hand the build to it with the source root and the compiler path. That script owns signing and installation into wherever the product keeps its binaries. Without that file - which is the normal case, and the one this book assumes - `Build.pb` just builds into `bin/` as above. The details are in the build reference page; the important part is that this is a developer hook for building THE TOOL, and it changes nothing about model generation.

Your first model

Two tiny models, one command, one build, three requests - and an answer you can check by hand.

Two models that cost nothing to get

You do not need to find a model to try this. `examples/CreateDemoModels.pb` carries two of them as bytes in a data section and writes them to disk. Run it once in the IDE:

```
Created output/twice.onnx: y = x + x, shape [1,N].
Created output/dense.onnx: y = x * diag(2,3), shape [1,2].
These are ONNX IR 8 / opset 20 model fixtures, not executable code.
```

They are real ONNX protobuf files, not a private format that looks like one. No package parses them into existence and nothing is downloaded - the bytes are literally in the source, because a first example that requires you to obtain something first is not a first example.

The two do different jobs.

`twice.onnx` is one `Add` node computing `y = x + x`, with the input shaped `[1,N]`. That `N` is a named dimension, not a number, so this model has a **runtime dimension**: the length is decided per request. It is the one that demonstrates the resident lifecycle.

`dense.onnx` is one `MatMul` against a constant 2-by-2 diagonal matrix holding 2 and 3, with the input shaped `[1,2]`. Every extent is known, so this is a **fixed-shape** model, and it has a real weight to pack. It is the one that demonstrates precision choices, because there is something to reduce.

> **Note.** The tool tells you which is which. Ask it:

```
> .\bin\PureMetalOnnxCompilerCLI.exe --inspect output\twice.onnx
Model:        output/twice.onnx
Bytes:        108
IR version:   8
Graph:        twice
Nodes:        1
Initializers: 0 (raw=0, typed=0, external=0)
Value infos:  0
Inputs:
  x type=1 [1,N]
Outputs:
  y type=1 [1,N]
Operators:
  Add 1
```

`[1,N]` with a letter in it is the runtime dimension. `type=1` is ONNX's number for float32. Run the same command on `dense.onnx` and you get `[1,2]`, one initializer, and `MatMul 1`.

Generate the Windows source

From the checkout root, one command:

```
> .\bin\PureMetalOnnxCompilerCLI.exe --compile output\twice.onnx --output output\twice --target windows
PASS: reusable source emitted to ...\output/twice.pb
Weights: ...\output/twice.pmw
Model requires one downstream build; requests use runtime dimensions.
```

`--output` is a file PREFIX, not a directory and not a filename. It names the stem that four things are built on, and the directory it sits in must already exist:

```
> ... --output output\nodir\twice --target windows
ONNX COMPILER ERROR: output folder does not exist: ...\output/nodir/
```

The tool will not create the directory for you, because a mistyped prefix that quietly creates a folder is harder to notice than one that stops.

> **Trap. Check the exit code in scripts.** Every refusal in this book exits

nonzero, but a script that ignores that will happily go on and build the source from the LAST successful generation. In PowerShell: `if ($LASTEXITCODE -ne 0) { throw 'Model generation failed' }`. Using stale output after a failed run is the single easiest way to spend an hour debugging a change you never actually generated.

The same thing, in the window

If you prefer, run `bin\PureMetalOnnxCompiler.exe` instead. Choose the model, the output prefix, the Windows target and FP32 precision, and start generation. The window runs the CLI beside it and shows you what it said - the same text, the same result, the same files.

Build the consumer and run it

Generation is done. Now it is an ordinary application build.

`examples/RunTwiceWindows.pb` is a complete consumer of the model you just emitted. It includes the generated source directly:

```
XIncludeFile "../output/twice.pb"
```

Open it in the IDE, enable **thread-safe executable** and the **optimizer**, select the x64 native backend, and run it. This build is yours; the ONNX tool played no part in it.

```
PASS request 1: 3 values doubled by one resident model.
PASS request 2: 6 values doubled by one resident model.
PASS request 3: 9 values doubled by one resident model.
All requests passed; model closed. No inference runtime DLL was loaded.
```

What actually happened in those three lines

This is a small program and it is worth reading, because its shape is the shape of every consumer you will ever write against this tool.

```
If PmModelInitialize(root+"twice.pmw")=0     ; ONCE: load and check weights
  PrintN("Model initialization failed: "+DError)
  End 1
EndIf
baseline=DLive                               ; remember live storage now

For request=1 To 3
  n=request*3
  PmModelResetRequest()                      ; drop the previous request
  DError=""
  input=PmModelInput(0)
  If DShape(input,1,2,1,n)=0 : ... : EndIf   ; THIS request is [1,n]
  For i=0 To n-1 : PokeF(Dt(input)\Data+i*4,i+request) : Next
  If PmModelExecute()=0 : ... : EndIf
  output=PmModelOutput(0)
  ... check every value ...
  PmModelResetRequest()
  If DLive<>baseline : PrintN("Request storage was not released.") : ... : EndIf
Next

PmModelClose()
If DLive<>0 : End 1 : EndIf                  ; nothing left behind
```

Three requests of 3, 6 and 9 values. One initialization. The weights were read, checked and kept once, and the three different input lengths did not disturb them.

Two details in there are not decoration. `DShape` is what declares this request's dimensions - rank 2, extents 1 and n - which is exactly the freedom `[1,N]` bought. And `DLive` is the runtime's own count of live tensor storage: the example asserts it is back to its baseline after every request and back to zero after the close. A leak in a resident model is not a slow leak; it is a leak per request.

> **Note.** The example checks every output value against `2*(i+request)`, not

just the first one, and not a tolerance. For an Add of float32 values that small, the answer is exact and the test should say so. Chapter 8 is about where tolerances are the right instrument and where they are an excuse.

Now try a packed weight

`twice.onnx` has no weights at all, so asking for INT8 there can only disappoint. `dense.onnx` has an eligible constant matrix:

```
> .\bin\PureMetalOnnxCompilerCLI.exe --compile output\dense.onnx --output output\dense-int8 --target windows --precision int8
PASS: generated ONNX source for windows (int8)
Source: ...\output/dense-int8.pb
Weights: ...\output/dense-int8.pmw (196 B)
Quantized weights: 1 (per-output-channel signed INT8, dynamic activation scales, INT32 accumulation)
```

```
Arena: 192 B including bounded inputs/outputs
Manifest: ...\output/dense-int8.json
Resident model: bind weights/arena once, fill inputs and execute repeatedly, then unbind when finished.
```

Read those lines carefully, because they are the tool being precise rather than enthusiastic. **One** weight was quantized. The scheme is named. The accumulation is INT32, not INT8. And the arena figure is what this model needs, inputs and outputs included.

> **Trap.** A 196-byte pack for an INT8 model where the FP32 pack was 144 bytes

is not a bug and not a failure of quantization. At this size the pack's header, scales and alignment dominate everything else. Compression is a property of large weight tensors; on a 2-by-2 matrix there is nothing to compress. Draw conclusions about size from your real model's manifest, never from a demo.

Where to go from here

Use different output prefixes when you are comparing things. `dense-int8` and `dense-fp32` sitting side by side can be diffed; two generations into the same prefix cannot, and a failed second generation next to a successful first one is a trap you set for yourself.

Then read chapter 4, which is about what the compiler will accept from a model that you did not write yourself.

Reading a model

What the compiler accepts, what it turns down, and exactly how the refusal reads.

Look before you compile

`--inspect` parses the file and tells you what is in it, without emitting anything. It is the cheapest thing you can do to a model you did not write, and it answers most of the questions the rest of this chapter is about.

```
> .\bin\PureMetalOnnxCompilerCLI.exe --inspect output\dense.onnx
Model:        output/dense.onnx
Bytes:        138
IR version:   8
Graph:        dense
Nodes:        1
Initializers: 1 (raw=1, typed=0, external=0)
Value infos:  0
Inputs:
  x type=1 [1,2]
Outputs:
  y type=1 [1,2]
Operators:
  MatMul 1
```

The operator histogram at the bottom is the part to read first. Compare it against the list in the next section and you know in seconds whether this model has any chance at all.

> **Note.** `external=0` matters on a large model. An initializer stored externally is a reference to another file, not data. Check that number before you copy a model somewhere and wonder why it stopped working.

The operator subset, in full

This is the complete list the fixed-shape validator implements, with Constant, which both paths turn into an initializer before planning; the runtime-dimension path adds Shape and ConstantOfShape, and the control-flow and sequence operators marked below. There is no long tail and no partial support hiding behind a flag - an operator is on this list or it is refused.

```
Arithmetic      Add  Sub  Mul  Div  Pow  Mod  Min  Max  Sum  Mean  Reciprocal
Activations     Relu  LeakyRelu  PRelu  Sigmoid  Tanh  Clip  Elu  Selu  Celu
                HardSigmoid  HardSwish  Softplus  Softsign  Mish  Gelu  Swish
                ThresholdedRelu  Shrink  Hardmax
Elementwise     Exp  Log  Sqrt  Abs  Neg  Sin  Cos  Tan  Asin  Acos  Atan
                Sinh  Cosh  Asinh  Acosh  Atanh  Floor  Round  Ceil
                Sign  Erf  IsNaN  IsInf
Bitwise         BitwiseNot  BitwiseAnd  BitwiseOr  BitwiseXor
Linear algebra  MatMul  Gemm  Conv  ConvTranspose  DeformConv  LSTM  RNN  GRU
                Einsum  Det
Normalization   Softmax  LogSoftmax  LayerNormalization  BatchNormalization
```

```
                InstanceNormalization  GroupNormalization  LpNormalization
                MeanVarianceNormalization  LRN  RMSNormalization
Detection       NonMaxSuppression  RoiAlign
Losses          NegativeLogLikelihoodLoss  SoftmaxCrossEntropyLoss
Quantized       QuantizeLinear  DequantizeLinear  DynamicQuantizeLinear
                MatMulInteger  QLinearMatMul  ConvInteger  QLinearConv
Pooling         MaxPool  AveragePool  LpPool  GlobalMaxPool  GlobalAveragePool
                GlobalLpPool  MaxUnpool  MaxRoiPool
Reductions      ReduceMean  ReduceSum  ReduceMax  ReduceMin  ReduceProd  ReduceL1
                ReduceL2  ReduceSumSquare  ReduceLogSum  ReduceLogSumExp
                ArgMax  ArgMin  CumSum  CumProd
Signal          STFT  DFT  HannWindow  HammingWindow  BlackmanWindow
                MelWeightMatrix  Resize  Upsample  GridSample  AffineGrid
Comparison      Equal  Greater  GreaterOrEqual  Less  LessOrEqual  And  Or  Xor
                Not  Where
Indexing        Gather  GatherElements  GatherND  Slice  Expand  Pad  NonZero
                ScatterND  ScatterElements  Scatter  TopK  Range  OneHot  Trilu
                Compress  ReverseSequence  Unique       (Unique: runtime dimensions)
Shape           Reshape  Flatten  Squeeze  Unsqueeze  Transpose  Concat  Split
                Tile  DepthToSpace  SpaceToDepth  Size  EyeLike  Col2Im
                CenterCropPad
Transformer     Attention  RotaryEmbedding  TensorScatter  LinearAttention
                CausalConvWithState
Other           Cast  CastLike  BitCast  Identity  Dropout (inference)
Random          RandomNormal  RandomNormalLike  RandomUniform  RandomUniformLike
                Bernoulli  Multinomial
Control flow    If  Loop  Scan                        (runtime dimensions)
Sequences       SequenceEmpty  SequenceConstruct  SequenceInsert  SequenceAt
                SequenceErase  SequenceLength  SplitToSequence
                ConcatFromSequence  SequenceMap
                                                      (runtime dimensions)
```

Anything else is turned down by node number and by name:

```
ONNX COMPILER ERROR: node 0 uses unsupported operator TfIdfVectorizer
```

The operators added in September 2026 - from Mod to Dropout in the list, and in a second group the trigonometric and hyperbolic functions from Tan, the bitwise operators, Hardmax, RNN, GRU, Det, the normalizations after InstanceNormalization, NonMaxSuppression, RoiAlign, Upsample, Compress, ReverseSequence and EyeLike, and after them GridSample and the quantized operators - share one kernel source for every target (`runtime/tensor_ops.pmi`), so they give the same bits on Windows, the Pi 4, the UNO Q, the Pico and the Pico 2; chapter 8 says how that is checked.

> **Note.** The quantized operators are the ONNX operators a model that was

quantized before export already contains; they carry UINT8, INT8 and INT32 tensors, which Cast also converts to and from. They are not the `--precision int8` mode later in this chapter, which quantizes an FP32 model's weights itself.

> **Trap.** Being on the list is necessary, not sufficient. An operator is

supported for the attribute combinations, ranks and element types that have actually been implemented and proved. A `Conv` with a grouping this runtime does not carry is still refused, and it is refused at validation rather than accepted and silently ignored. Read the error, not the list.

> **Trap.** That is the rule. The official ONNX node tests (chapter 8) found

eight forms that were accepted and computed the wrong answer; all eight are fixed, each recorded in its forum topic (848 and 850 to 856).

Forms fixed since the node tests first ran, and the limits each keeps: - Clip reads a min or max that arrives while the program runs from its tensor when the node runs; both bounds must be FLOAT scalars. - LayerNormalization normalises over any axis. Scale and B must have X's extent on every normalised axis and 1 before it; one that broadcasts inside the normalised block is refused. - Softmax normalises along any axis; below opset 13 it flattens every axis from axis onward, as Softmax-11 specifies. - MatMul promotes a 1-D operand the way numpy.matmul does, on its own or against a batched operand. - Pow reads each operand in its own element type. An INT64 base gives an exact integer power; a FLOAT exponent that is not a whole number, a negative exponent on a zero base, or an unsigned exponent above the signed range stops the program at that node rather than rounding. - ScatterND computes its reductions add, mul, max and min, on FLOAT and INT64 data (and INT32 on the runtime-dimension path). - BatchNormalization with training_mode 1 normalises with the batch's own statistics and needs all three outputs; BatchNormalization-9's training outputs and spatial 0 are refused.

Attributes: the value decides, not the presence

Exporters write attributes out even when they hold the default, so a model full of `saturate = 1`, `auto_pad = NOTSET` and `stash_type = 1` is normal. None of those is a reason to refuse. An attribute is refused only when its value asks for something the kernel does not compute, and the refusal names the operator, the node, the attribute and the value, on either path:

```
ONNX COMPILER ERROR: Resize node up: nearest_mode is absent, so it takes the specification's default
round_prefer_floor, which runtime-dimension emission does not implement for mode nearest; floor is.
```

That one is worth reading twice. The attribute is not in the model at all, and the model is still refused, because absent means the specification's default and the default is a rounding this runtime does not implement. Presence was never the question; the value is.

What each operator with attributes accepts, briefly - the validation page has the full table:

```
Cast        saturate with any value (it only affects float8 casts)
Shape       start and end, negative and out-of-range values included
LSTM        layout 0, input_forget 0, the default activations
LayerNorm   stash_type 1
Conv        auto_pad NOTSET, VALID, SAME_UPPER, SAME_LOWER
ConvTransp. the same auto_pad values; not output_shape
Resize      nearest (floor) or linear; half_pixel or asymmetric;
            antialias 0, exclude_outside 0; attributes those modes ignore
ScatterND   reduction none, add, mul, max, min
ScatterEl.  reduction none, add, mul, max, min; negative indices
TopK        axis, largest, sorted (sorted order is written either way)
ReduceMax   keepdims, noop_with_empty_axes; axes input or attribute
ReduceProd  the same as ReduceMax
InstanceN.  epsilon
```

```
Pad         mode constant, reflect, edge, wrap; the axes input
ReduceSum   noop_with_empty_axes, when the axes are given
```

Note. auto_pad SAME needs the input width, which a runtime-dimension model

only knows while it runs, so that form is emitted as its own call - `DConvSame(...)` rather than `DConv(...)` - that computes the padding and then runs the same convolution. A model that uses only the default forms emits exactly the calls it always did.

An opset import of another domain that no node uses, such as `ai.onnx.ml`, changes nothing and is accepted. A node that actually lives in another domain is refused, named, on either path:

```
ONNX COMPILER ERROR: node 0 uses unsupported domain com.microsoft
ONNX COMPILER ERROR: FusedGelu node gelu: its domain com.microsoft is not implemented by runtime-dimension
emission, which implements the ai.onnx domain only.
```

The first is the fixed-shape path, the second the runtime-dimension path.

The opset range it will admit

```
> ... --compile dense28.onnx --output out\d --target windows
ONNX COMPILER ERROR: ai.onnx opset 28 is outside the audited range 7..27

> ... --compile dense5.onnx --output out\d --target windows
ONNX COMPILER ERROR: ai.onnx opset 5 is outside the audited range 7..27
```

Both ends are refusals. Too new is obvious; too old is the one that surprises people, and it is the same argument in reverse. Operator semantics changed across those early versions, and a compiler that assumes modern behaviour while reading an old graph produces plausible numbers that are wrong.

"Audited" is the honest word. It means somebody checked what each operator means in that range, not that the range was chosen to look generous.

Above opset 20, every version of every operator the compiler implements was compared with the one before it in the ONNX schema history, and a private gate holds that audit to the installed onnx package. Most of those versions only admit more element types, which the compiler refuses anyway, so the operator is accepted through opset 27. An operator whose meaning changed has a ceiling, and a model that imports an opset past it is refused by name:

```
ONNX COMPILER ERROR: node 0 (GroupNormalization): the model imports ai.onnx opset 21, where GroupNormalization
is GroupNormalization-21 or later, which scales and shifts per channel, not per group; this compiler
implements GroupNormalization through opset 20.
```

Trap. The runtime-dimension path is narrower at the old end, and it says so

in its own words. A model with a symbolic extent goes down a second emitter that takes each node, subgraph nodes included, from its operator's floor - the oldest opset whose definition of it is the one implemented - through its ceiling, opset 27 at most:

```
ONNX COMPILER ERROR: The model imports ai.onnx opset 28; runtime-dimension emission implements operator
definitions through opset 27.
ONNX COMPILER ERROR: Softmax node (unnamed): the model imports ai.onnx opset 11, whose definition of Softmax
differs from the one runtime-dimension emission implements, which is current from opset 13 through 27.
```

The second one is the reason for floors. Softmax at opset 11 flattens its input to two dimensions around the axis; from 13 it works along one axis. Same name, different operation, and a model that asks for the old one gets a sentence instead of the new one's numbers.

Most operators on that path start at opset 11 or earlier; GreaterOrEqual starts at 12, Softmax at 13, LayerNormalization and STFT at 17, and an operator not yet checked across the versions still needs 20. InstanceNormalization, TopK, ScatterElements, ReduceMax, ReduceProd, Not, Identity and Pad have floors of their own - TopK from 10, ScatterElements and Pad from 11, InstanceNormalization from 6, the rest from 1 - and a form that belongs to another opset, such as Pad's wrap mode at opset 18, is refused by name. The operators added in September 2026 each start at the opset whose definition they compute (Mish at 18, Gelu at 20, Trilu and HardSwish at 14, GridSample at 16, DFT and the windows at 17, the bitwise operators, GroupNormalization, Col2Im and CenterCropPad at 18, DeformConv at 19, AffineGrid at 20, RMSNormalization, RotaryEmbedding and Attention at 23, Swish and TensorScatter at 24, CumProd and BitCast at 26, LinearAttention and CausalConvWithState at 27, most of the rest at 11 or earlier), and LogSoftmax and Hardmax before 13 are accepted only on the last axis, where the old flattening definition and the new one agree. Two have a ceiling as well: GroupNormalization up to 20 (from 21 its scale is per channel, not per group), and Upsample up to 9, where it was deprecated for Resize - and only with fixed shapes; MelWeightMatrix only with its five inputs as initializers, from which the compiler computes the matrix; Scatter, the same, is read at opsets 9 and 10 as ScatterElements without a reduction. The fixed-shape path admits 7 through 27. The two paths are not equally broad at the old end, and the message tells you which one you are on.

Fixed shapes and runtime dimensions

Every extent in an ONNX graph is either a number or a name. That single distinction decides which of the two emitters runs and which API you get.

All numbers is a fixed-shape model. Every buffer size is known at generation time, lifetimes are known, and the arena is planned exactly. You get the `PmOnnx...` API, which never allocates: you hand it memory and it uses that memory.

Any name is a runtime-dimension model. The extent is decided per request. You get the `PmModel...` API, which separates the model's persistent state from each request's temporary tensors.

`twice.onnx` from chapter 3 is `[1,N]`, so it is the second kind, and that is what let one bind serve requests of 3, 6 and 9 values.

A graph with all-number inputs also takes the runtime-dimension path when a node's output is not declared at all, with no type and no shape - the intermediates of a function-expanded graph are like that. The fixed-shape planner needs every value's size up front, and the runtime-dimension path works it out as the program runs. A value that IS declared, with a name for an extent, is a different matter; the next section shows how that one is refused.

Pinning a dimension with --shape

You can turn a symbolic extent into a number, and get the fixed-shape path for a model that did not ship that way:

```
--shape x=1,4
```

It is checked, not trusted. Name an input that is not there, or an extent that is not a positive integer, and it stops:

```
ONNX COMPILER ERROR: --shape names no graph input: z
ONNX COMPILER ERROR: --shape extents must be positive decimal integers for input x
```

Trap. Pinning the inputs does not always pin the graph, and this is the

refusal that catches people out. On `twice.onnx`, `--shape x=1,4` produces:

```
ONNX COMPILER ERROR: tensor y still has a dynamic extent
```

The input is fixed now, but the model's declared OUTPUT still carries the symbol `N`, so the graph as written is not fully determined. The compiler will not quietly decide that `N` must be 4 because the input happened to be. It names the tensor that is still open and stops. Fix the model's declared shapes, or use the runtime-dimension path as intended.

Control flow and sequences

`If`, `Loop` and the sequence operators compile on the runtime-dimension path, for every target. A model that contains one of them takes that path even when every extent in it is a number: how many times a loop runs, and which branch an `If` takes, are decided while the program runs, and the fixed-shape planner has nowhere to put that.

A branch or a loop body is its own small graph. It can read any value its enclosing graphs have already computed, by name, and the generated program gives each body its own procedure. A name a body defines that the model also defines somewhere else is renamed before anything is emitted, so the body always reads its own definition.

Kitten TTS nano uses both. Its duration step is a `Loop` that turns token durations into frame indices one token at a time, building a sequence it joins at the end; its two `If` nodes squeeze an axis only when it has extent 1, so the branches return tensors of different rank. That is fine: every value on this path carries its shape at run time.

An `Identity` whose input is a sequence (opset 14 and later) hands the sequence on, or copies it when something reads it again.

`Scan` (opset 9 and later) and `SequenceMap` (opset 17 and later) are compiled as the `Loop` they describe. A `Scan` runs its body once per slice of its scan inputs; each slice is a `Gather` at the iteration number (for a reversed input, at the last index minus the iteration number), and each scan output is the loop's stacked output, reversed and moved to its axis when the attributes ask. A `SequenceMap` reads one element of each input sequence per iteration and inserts each result into its output sequence, as the operator's ONNX function body does. A negative scan axis needs the input's rank declared, and a scan output on another axis than 0 needs the body output's rank declared. `Scan-8`, with its batch axis and `sequence_lens`, is refused by name.

A sequence is a value with its own storage: one block holding every element's shape and data. On the bare-metal targets that block comes out of the same arena you bind for the model, so a sequence that outgrows the arena is refused in a sentence rather than truncated. A loop that appends one element per iteration does not copy the sequence each time; the compiler hands the block from one iteration to the next when nothing else reads it.

If a model takes or returns a sequence, you build or read it with its own calls, because a sequence has no single shape to give `DAlloc`:

```
id = PmModelInput(0)
DSeqReset(id, 1)                            ; an empty sequence of FLOAT
address = DSeqAppendShape(id, 1, 2, @dims(0)) ; one element; fill it
...
PmModelExecute()
out = PmModelOutput(0)
count = DSeqCount(out)                      ; then DSeqItemRank, DSeqItemDim,
                                            ; DSeqItemBytes, DSeqItemData
```

What is refused, in the tool's words:

```
ONNX COMPILER ERROR: Loop node (unnamed) has neither a trip count M nor a condition; the ONNX specification
defines that form as a loop that never ends, so it is refused.
ONNX COMPILER ERROR: Loop node (unnamed) scan output 0 is a sequence; Loop scan outputs are tensors stacked
along a new first axis.
ONNX COMPILER ERROR: SequenceEmpty node (unnamed) attribute dtype=10 is not FLOAT (1), INT32 (6), INT64 (7) or
BOOL (9), the element types a sequence carries here.
ONNX COMPILER ERROR: SplitToSequence node (unnamed) attribute keepdims=2 is not 0 or 1.
ONNX COMPILER ERROR: ConcatFromSequence node (unnamed) has no axis attribute; the operator requires one.
ONNX COMPILER ERROR: Graph input 'opt_seq' is an optional value; optional types (Optional, OptionalHasElement,
OptionalGetElement) are not implemented.
```

Some things can only be refused while a request runs, and they are, with the node number:

```
Node 1: SequenceAt position 3 is outside [-3, 2] for a sequence of 3 elements.
Node 0: A Loop ran zero iterations and its scan output does not declare every extent, so its empty result has
no defined shape.
```

> **Note.** The second one is not fussiness. ONNX does not say what shape a scan

output has when the loop never ran, and two runtimes can answer differently. Declare the body output's shape in full and the answer is defined: `[0]` followed by those extents.

Random operators, and checking a model that has them

`RandomNormal`, `RandomNormalLike`, `RandomUniform`, `RandomUniformLike`, `Bernoulli` and `Multinomial` are compiled on both paths and for every target. They are the one family where the ONNX specification leaves something open that a compiler must not: it fixes the distribution, the element type and the shape, and says nothing about the generator. Two runtimes that both follow the specification can therefore disagree on every sample. This compiler specifies the generator (chapter 8 states the contract in full), so one seed, one request number and one shape give the same FLOAT values on Windows, the Pi 4, the UNO Q, the Pico and the Pico 2.

What is claimed: FLOAT output (`dtype` 1, or no `dtype` on a Like node whose input is FLOAT); `mean` and `scale`, or `low` and `high`, with their specification defaults; the `seed` attribute; `shape` of up to eight axes; up to 4,294,967,295 elements per tensor. A random node is never folded and never scheduled with the bind-time constant work: every execution draws again. `Bernoulli` (opset 15) takes FLOAT probabilities and writes FLOAT, UINT8, INT8, INT32, INT64 or BOOL; it writes 1 where the node's uniform draw is below the probability, as the operator's text says (its ONNX function body compares the other way

round). `Multinomial` (opset 7) takes FLOAT rows of unnormalized log-probabilities and writes INT32 or INT64 class indices, `sample_size` per row. Every other form is refused with the node, the operator, the attribute and the value:

```
ONNX COMPILER ERROR: node 0 (RandomNormal) attribute dtype = 10 (FLOAT16) is not implemented: this compiler
generates FLOAT (dtype 1) random tensors only. Re-export the node with dtype 1, or follow it with a Cast.
ONNX COMPILER ERROR: node 0 (RandomUniformLike) has no dtype attribute, so its output takes its input's
element type INT64 (7); this compiler generates FLOAT (1) random tensors only. Add dtype = 1 to the node.
ONNX COMPILER ERROR: node 0 (RandomNormal) has attribute foo, which the ONNX specification does not define for
RandomNormal; nothing was emitted.
ONNX COMPILER ERROR: node 0 (RandomNormal) attribute shape extent 1 = -3 is negative.
ONNX COMPILER ERROR: node 0 (RandomNormal) attribute mean = +Infinity is not a finite number; the generator
needs a finite mean.
```

On the runtime-dimension path a Like node's input type is sometimes only known when a request arrives. A dtype-less Like node fed a non-FLOAT tensor then fails that request, in the same words:

```
Node 1: RandomUniformLike has no dtype attribute, so its output would take the input's element type 7; only
FLOAT (1) random tensors are generated. Add dtype=1 to the node.
```

Trap. A reference runtime does not reproduce its own noise, so a model with

random nodes cannot be compared sample for sample against it as compiled. Its output is a different draw from the same distribution, and every error metric computed against the reference measures the noise, not the compiler.

That is what `--random-inputs` is for. It compiles every random node as an extra model input that you fill, so you can feed the reference's own noise (add the random nodes' outputs to the reference model's graph outputs, run it once, and pass those tensors in) and get a sample-exact comparison:

```
--random-inputs makes every RandomNormal/RandomUniform(-Like), Bernoulli and Multinomial node an extra model
input the caller fills.
```

The window has the same switch, "Random nodes as model inputs (comparison)". The extra inputs follow the graph's own inputs in node order, and the manifest's `random.nodes[].input_index` says which is which. Each supplied tensor must have the element type (`random.nodes[].dtype`) and the shape the node would have produced, and anything else fails the request with a sentence rather than flowing on. The option is refused for a model with no random node:

```
ONNX COMPILER ERROR: --random-inputs was given, but the model has no RandomNormal, RandomNormalLike,
RandomUniform, RandomUniformLike, Bernoulli or Multinomial node to turn into an input. Remove the option.
```

Trap. `--random-inputs` CHANGES THE MODEL'S INPUTS. A program built for the

generated model will not drive the comparison build, and a comparison build generates nothing: leave it running with zeros in those inputs and the model runs noiseless. Use it to check the compiler against a reference, then compile again without it for anything you ship.

Errors are sentences

Every refusal in this chapter is a complete statement of what was wrong. Nothing prints a bare code. Compare these two, which are the same class of mistake at two different layers:

```
ONNX COMPILER ERROR: ONNX model does not exist or is empty: output/nosuch.onnx
ONNX COMPILER ERROR: length-delimited field exceeds its enclosing message
```

The second is the bounded protobuf reader refusing a truncated file. It does not read past the end of a span and then discover the problem later; it checks the length against the message it is inside, and stops there. That check is the one thing standing between a malformed model and whatever the parser would otherwise have done with it.

> **Note.** Format checks are not a security audit. They stop a malformed file

from confusing the parser. They do not make an untrusted model safe, and weights from an unknown source are still weights from an unknown source.

Precision: five choices, two mechanisms

`--precision` takes `fp32`, `int8`, `fp16`, `bf16` or `int4`, and the default is `fp32`. Anything else is named right back at you:

```
ONNX COMPILER ERROR: --precision must be fp32, int8, fp16, bf16, or int4
```

Underneath, there are only two different things happening.

FP32 is the baseline. Weights and arithmetic are binary32 throughout. This is the one to establish correctness against. Every claim anyone makes about a reduced precision is a claim relative to this.

INT8 changes the arithmetic, selectively. Eligible constant linear weights become per-output-channel signed INT8; activations are quantized at run time, one scale per row; products are integers summed in INT32; the boundaries stay FP32. The tool tells you how much of your model that touched:

```
Quantized weights: 1 (per-output-channel signed INT8, dynamic per-row activation scales, INT32 accumulation)
```

If that count is zero, INT8 did nothing for this model, and no amount of re-running it will change that. Operators outside the eligible set keep their FP32 path in the same program.

What INT8 actually runs, and what it does not

The count above says how many weights were quantized. It does not say how each one is computed, and that is worth knowing before you read a timing or an error figure.

The eligible set is linear weights, and it is not everything. Constant weights of convolution, the recurrent layer's two weight matrices, general matrix multiply and the general matrix-multiply-with-bias operator are eligible. Transposed convolution is not: its operator requires a float weight and refuses any other kind, so those weights stay FP32 in an INT8 build by construction rather than by omission. Everything that is not a linear weight - element-wise arithmetic, the activations, normalisation, the transforms, the shape operators - is FP32 and untouched.

The tensors between operators stay FP32, and each operator quantizes its own input one row at a time. A row is what one output is summed over: a token for matrix multiply and the matrix-multiply-with-bias operator, one input position across every channel for a convolution, one step for the recurrent layer. Each row gets its own scale from its own largest magnitude, so one loud token or one loud moment of audio no longer flattens the resolution of every other one - which is exactly what a single scale per tensor did, and why the scheme changed.

Some weights are wide. A model can carry a measured precision plan naming weights that need more than eight bits. A wide weight is stored as two INT8 planes - the weight, and its rounding error on a grid 254 times finer - and its activations are quantized to 16 bits; the arithmetic is still integer products and integer sums. Today exactly one model has a plan: the pinned speech model of chapter 7, whose text, duration and pitch front end and three nodes next to the waveform (59 of its 183 quantized weights, about 8% of its multiply-accumulates) are wide. Every other model is compiled narrow. A runtime-dimension manifest says how many weights are wide (`wide_weight_count`).

The arithmetic is defined exactly, and every target computes the same bits. A row's maximum is taken on the bit patterns of the magnitudes; a row below 1e-30 is a row of zeros; the scale is maximum/127 (32767 when wide) and each value is rounded half to even after one multiply. Each output starts at zero and adds, once per kernel tap and once per 512 reduction elements when wide, the exact integer sum converted to FP32 times the row's scale; then it is multiplied by the weight's channel scale (for a wide weight, the two planes are joined as F1 + F2/254 first), and a convolution adds its bias last. Every one of those steps is a single binary32 operation, so the Windows kernels, the 64-bit Arm vector kernels and the Cortex-M kernels give **identical output bits for identical input**. Chapter 8 says how that is checked.

A NaN or an infinity in an INT8 operator's input is refused. The row maximum sees it first; the operator writes nothing and the request fails at that node, in a sentence, on every target and on both model interfaces.

Convolutions of any stride take the vector kernel. The input is quantized into one row per phase of the stride, so the sixteen outputs a vector step computes always read sixteen neighbouring values. A strided convolution used to fall back to a slow scalar body; it no longer does.

Quantizing the activations costs one pass for the row maxima and one multiply-and-round pass. On a convolution that is a small fraction of the work; on a tiny operator it can be most of it.

> **Trap.** INT8 is not automatically faster. It moves a quarter of the weight

bytes, and on a processor without an integer dot-product instruction its products are not cheaper to issue than a well-blocked FP32 kernel's. Whether it wins depends on the kernel and the processor. Measure your own model on your own target; do not reason from the weight-file size.

Measured: the speech model at INT8

The full speech model of chapter 7 at `--precision int8` , built for Windows and run on a desk machine (Intel i9-14900HX), scored against ONNX Runtime running the same pinned FP32 graph. "Durations changed" counts the tokens whose predicted length in frames differs from FP32's; the log-mel figure is the mean distance between the two spectrograms after aligning them, in dB.

```
reference input (143 tokens)     durations changed   log-mel vs ONNX Runtime FP32
  this tool, FP32                    0 of 143            0.294 dB  (the floor)
  this tool, FP16 weight storage     0 of 143            0.442 dB
  this tool, INT8                    0 of 143            0.588 dB
  the earlier INT8 scheme            9 of 143            about 5.2 dB
```

```
a 295-token sentence, INT8          0 of 295            0.647 dB
                  FP16 storage      1 of 295            1.835 dB
a 69-token sentence, INT8           0 of 69             0.442 dB
```

Speed on the same machine, the two builds alternated request by request because the processor throttles: the reference request took **5.51 s at FP32 and 4.47 s at INT8** (medians of five rounds): INT8 is 1.23 times faster. The rest of the model - element-wise arithmetic, the transforms, the transposed convolutions - is FP32 in both builds.

Those two numbers are from before the Windows runtime used every processor (chapter 5, "Threads on Windows"). On all 32 logical processors of the same machine, five interleaved rounds against the previous build: **1.39 s at FP32 and 1.24 s at INT8** (medians), where the previous build took 6.55 s and 5.09 s in the same rounds - and every waveform bit-identical to the previous build's. With `--threads 1` the same request takes 11.2 s and 10.1 s; the curve through 2, 4, 8 and 16 threads is in `docs/VALIDATION.md`.

The Pi 4 and UNO Q (Advanced SIMD) and the Pico and Pico 2 run the same INT8 arithmetic bit for bit (chapter 8). The speech model does not fit a Pico. **It has not yet been measured on Pi 4 silicon with this scheme**; the table below is the earlier scheme's board run, kept because it is why the scheme changed.

The earlier scheme on the Pi 4, and why it was replaced

On 2026-09-16 the same model, with one activation scale per whole tensor, the unit-stride-only vector convolution and three activation passes per operator, ran on the Pi 4 on four cores:

```
request 2 ("You are hearing a complete sentence produced on this
small computer.")            FP32 1 core   FP32 4 cores   INT8 4 cores
  total                          88.079 s       34.614 s    72.782 s
  Conv           (88 nodes)      70.450         17.149      51.500
  MatMul        (102 nodes)       2.783          2.784       7.250
  Gemm           (73 nodes)       0.378          0.378       0.015
```

It was 2.10 times slower than FP32 on the same four cores, and 9 of 143 durations changed, so the audio was differently timed and judged worse by ear. The kernels that ran then are gone: the scheme above replaced the per-tensor scale (the durations) and the vector kernels were rewritten around it (the speed). The replacement's Pi 4 run is pending, and until it exists the honest statement is the desk table above plus the instruction counts in chapter 8.

> **Note.** For a precision change the acceptance test is still a person

listening. Durations and spectrogram distance say the INT8 build predicts the same timing as FP32 and sounds close to it; they do not replace an ear.

INT8 output is **not** bit-identical to FP32 output and is not meant to be. What IS bit-identical is INT8 against INT8: the same model on every target computes each INT8 operator's output to the same bits.

FP16, BF16 and INT4 change storage only. The weights are stored narrow and **decoded once, at bind time, into resident FP32 memory**. The arithmetic afterwards is the FP32 arithmetic. BF16 trades mantissa bits for FP32's exponent range, which matters for weights with a wide dynamic range; FP16 does the opposite; INT4 is the aggressive one and carries scales.

Trap. These three shrink the FILE. They do not shrink the RAM the model

needs while it is running, because the decode target is full-width FP32 and it is resident for the model's whole life. On a microcontroller, where flash and RAM are two different budgets, this is exactly the distinction that decides whether a model fits - and choosing INT4 does not make a large model fit a small part. The manifest reports the decoded reserve as its own number, and chapter 5 shows where.

What to do with your own model

Start at FP32 with inputs whose answers you already know. Check the shapes and element types the tool reports against what you expected. Read the manifest's memory accounting and compare it with the memory the target actually has. Exercise your error paths - bind failures, allocation failures, a request larger than the arena - because a resident model that cannot fail cleanly will fail dirtily.

Only then compare a reduced precision on representative data, and compare it yourself. A successful generation says the graph was understood. It does not say the answer is good enough for what you are doing.

What comes out

Four things with one prefix, two APIs, and one rule: they travel together or not at all.

The set

One `--compile` produces four things, all named from the `--output` prefix:

```
output\dense-int8.pb          the emitted source
output\dense-int8.pmw         the checked weight pack
output\dense-int8.json        the manifest
output\dense-int8.runtime\    the execution sources this model needs
```

On a bare-metal target the source has that target's extension instead - `.pi4` for the Pi 4, `.unoq` for the UNO Q, `.pico`, `.pico2` - and the runtime folder holds different files. The shape is the same.

> **Trap.** These four are **one artefact in four files**. The source contains

the weight pack's CRC and identity; the pack has to be that exact file; the runtime folder is the version of the runtime this source was written against. Mix a source from one generation with a pack or a runtime from another and you will be told, but you will be told at bind time on the target rather than at your desk. Use a new prefix per generation and keep the set together.

The emitted source, from the top

```
; Generated by the PureMetal ONNX ahead-of-time compiler.
; Model: dense.onnx
; Backend: Host-language source for a native Windows x64 application
; Executable tensor code; no ONNX graph interpreter is present at runtime.
EnableExplicit
#PMO_USE_BATCHNORM = 0
#PMO_USE_STFT = 0
#PMO_USE_RESIZE = 0
#PMO_USE_CONVTRANSPOSE = 0
#PMO_USE_CONV = 0
#PMO_USE_LSTM = 0
#PMO_USE_INT8 = 0
XIncludeFile "dense-fp32.runtime/tensor_fp32_windows.pbi"
```

The `#PMO_USE_...` switches are the model saying what it needs. This graph is one `MatMul`, so convolution, the LSTM, the short-time Fourier transform and the rest are switched off, and the compiler underneath will not build code for them. On a part with 264 KiB of RAM that is not a micro-optimization; it is the difference between fitting and not.

Then the constants that describe the memory:

```
#PMO_WEIGHT_FILE_BYTES = 144      ; the pack's exact length
#PMO_WEIGHT_DATA_BYTES = 16       ; payload after the 128-byte header
#PMO_ARENA_BYTES = 128            ; working memory this model requires
```

```
#PMO_INT8_SCRATCH_OFFSET = 0
#PMO_INT8_SCRATCH_BYTES = 0
#PMO_STFT_SCRATCH_BYTES = 0
```

Use those symbols in your application. They are generated from the same plan the code was generated from, so they cannot drift from it; a number you copied into your own header can.

The weight pack, and why the source checks it

`.pmw` is a checked container: an eight-byte magic, a version, a 128-byte header, the payload, and a CRC-32 over the payload. The emitted source contains a validator that reads it byte by byte before accepting anything:

```
Procedure.i PmOnnxValidateWeights(*weights, bytes.i)
  If *weights = 0 Or bytes <> #PMO_WEIGHT_FILE_BYTES : ProcedureReturn 0 : EndIf
  ... magic, version, header length ...
  ... the model's own SHA-256 identity, word by word ...
  If PmTensorCrc32(*weights + 128, #PMO_WEIGHT_DATA_BYTES) <> $D793D46B
    ProcedureReturn 0
  EndIf
  ProcedureReturn 1
EndProcedure
```

Three separate things have to agree: the length, the identity of the model this pack was built from, and the CRC of the payload. The identity check is the one that earns its keep. A pack of the right length with the right CRC can still be the wrong model's weights, and inference against the wrong weights does not crash - it produces confident nonsense.

> **Note.** A CRC over hundreds of megabytes is not free. On a Pi 4, a full

CRC-32 pass over the pinned speech model's 324,621,440-byte pack was measured at about **1.95 seconds** with caches enabled, and vastly slower without them. That is a bind-time cost paid once, which is one more reason the model stays resident.

The manifest

The `.json` is the accounting. It is not needed at runtime; it is there so you can answer "will this fit" and "what exactly did I generate" without reading the source.


```
"precision": "int8",
"model_sha256": "71a76b66...",
"execution_contract": "resident-reusable",
"shape_contract": "fixed-dimensions-variable-input-values",
"weight_bytes": 196,
"arena_bytes": 256,
"decoded_weight_reserve_bytes": 0,
"reduced_storage_weight_count": 0,
"int8_scratch": {"offset":128,"bytes":128},
"ai_onnx_opset": 20,
"inputs":  [{"index":0,"name":"x","type":"float32","shape":[1,2],"bytes":8,"storage":"arena","offset":0}],
"outputs": [{"index":0,"name":"y","type":"float32","shape":[1,2],"bytes":8,"storage":"arena","offset":64}],
"quantization": {
```

```
  "mode": "int8",
  "scheme": "per-output-channel symmetric signed INT8 weights, dynamic per-row signed INT8 activations, INT32
accumulation, FP32 operator seams",
  "weight_count": 1, "fp32_weight_bytes": 16, "int8_weight_bytes": 4
},
"packed_tensors": [ {"name":"w","type":"int8","shape":[2,2],"offset":192,"bytes":4}, ... ]
```

Four fields answer the questions people actually ask.

`arena_bytes` is the working memory. `weight_bytes` is the pack. `decoded_weight_reserve_bytes` is what FP16, BF16 or INT4 will occupy after decoding - the number chapter 4's trap is about, reported separately so it cannot hide inside the storage saving. And `quantization.weight_count` tells you how much of your model INT8 really touched: two weights out of two hundred is a fact worth knowing before you draw a conclusion about speed.

`shape_contract` names which of the two emitters ran: `fixed-dimensions-variable-input-values` or `runtime-dimensions`.

The runtime folder

The tool copies out the execution sources the emitted model includes. They are not a library you installed; they are a snapshot taken at generation time and they belong to that generation.

```
dense-fp32.runtime\        (windows)
  tensor_dynamic_windows.pbi   tensor_fp32_windows.pbi   tensor_simd_windows.pbi
  tensor_pool_windows.pbi

dense-pi4.runtime\         (pi4 / unoq)
  math.pmi   tensor_fp32.pmi   tensor_fp32_neon.pmi
  tensor_trig_a64.pmi   tensor_dynamic_portable.pmi
```

Trap. An already-generated application does not pick up a runtime fix. Its

runtime is the copy sitting beside it. To adopt a change: rebuild the generator (chapter 2), regenerate the model's whole set into a fresh prefix, and build the application again. Copying one newer file into an old runtime folder is the worst of the available options, because the source and the runtime then disagree about a contract and nothing checks it.

Two APIs, and how to tell which you got

Fixed shapes give you `PmOnnx...`, and it allocates nothing.

```
PmOnnxBindMemory(weights, weightBytes, arena, arenaBytes)  ; check the result
PmOnnxInputAddress(index)      ; fill this buffer, declared type and shape
PmOnnxExecuteBound()           ; check the result
PmOnnxOutputAddress(index)     ; read before the next execution
PmOnnxLastErrorNode()          ; which node failed
PmOnnxUnbindMemory()           ; before you free anything
PmOnnxSetThreads(n)            ; Windows: at most n worker threads (see below)
```

You supply aligned, non-overlapping weight and working buffers, sized from `#PMO_WEIGHT_FILE_BYTES` and `#PMO_ARENA_BYTES`, and they must stay valid for the model's whole life. Binding does not take ownership of them. The bind

refuses overlap in either direction rather than trusting you.

Runtime dimensions give you `PmModel...`.

```
PmModelInitialize(weightPath)   ; Windows: reads the pack from disk
PmModelBindMemory(w,wb,a,ab)    ; portable: you supply both, as above
  PmModelResetRequest()         ; release the previous request's tensors
  PmModelInput(index)           ; a tensor ID; declare this request's dims
  PmModelExecute()              ; check the result and DError
  PmModelOutput(index)          ; read while its storage is live
PmModelClose()
PmModelSetThreads(n)            ; Windows: at most n worker threads (see below)
```

> **Trap.** `DError` is a string on Windows and an ASCII string POINTER on the

portable targets. The same field name, two different things, because the portable runtime has no string type to hand. Code that walks it assuming one will misbehave on the other. Chapter 3's example shows the Windows form.

A model with random operators

A model with a random node gains one procedure, a manifest entry and the generator's source in its runtime folder. A model without one gains nothing: its source closure is byte for byte what it was.

```
PmOnnxSetRandomSeed(seed)      ; fixed shapes: 1, or 0 for a seed wider than 32 bits
PmModelSetRandomSeed(seed)     ; runtime dimensions: the same
```

The model seed is 32 bits and starts at 0. On a 64-bit target any value whose upper half is all zeros or all ones is accepted, so `-1` and `$FFFFFFFF` are the same seed everywhere; anything wider is refused rather than truncated. A node with its own `seed` attribute keeps it and is not moved by the model seed.

Every execution that starts the graph uses the next **request number**: 0 after the model is bound, then 1, 2 and so on, so each request draws new values. Setting the seed restarts the numbering at 0, and binding again restarts it too and keeps the seed. The same seed and the same sequence of requests reproduce the same values, request for request.

```
"random": {"mode":"generated","set_seed_procedure":"PmOnnxSetRandomSeed",
  "model_seed_default":0,"model_seed_bits":32,
  "request_numbering":"0 after the model is bound or its seed is set; one more for every execution that starts
the graph",
  "generator":"threefry2x32-20", ...,
  "nodes":[{"index":0,"name":"","op":"RandomNormal","output":"a","mean":0.000000000,"mean_bits":"0x00000000",
            "scale":1.000000000,"scale_bits":"0x3F800000","seed":"model","dtype":1},
           {"index":1,"name":"","op":"RandomUniform","output":"b","low":0.000000000,"low_bits":"0x00000000",

"high":1.000000000,"high_bits":"0x3F800000","seed":"attribute","seed_bits":"0x40A00000","dtype":1}]}
```

`index` is the node's position in the graph, which is part of the key its values come from. With `--random-inputs` the mode is `inputs`, there is no seed procedure and no generator source, and each node carries the `input_index` you fill.

The runtime folder gains `tensor_random.pmi`, the generator - one source, included unchanged on every target - and the runtime-dimension wrapper for the target, `tensor_random_dynamic_windows.pbi` or `tensor_random_dynamic_portable.pmi`.

Threads on Windows

A Windows model runs on every logical processor its process may use. The count is taken when the model is bound, on the machine it is running on, not when it was compiled, and it is the process's own allowance, not the machine's: the affinity mask the process was started with (`start /affinity` , a job object's affinity, a parent's mask), every active processor of every processor group when the process has not been narrowed to part of one, and lower again when the process has a default CPU set. Started with `start /affinity 0xF` on a 32-processor machine, the same program uses 4 threads.

Binding takes the workers - `PmModelInitialize` and `PmOnnxBindMemory` do it - creating any that do not exist yet. `PmModelClose` and `PmOnnxUnbindMemory` park every one of them before they return: each is blocked on its own semaphore, runs nothing and holds no working buffer, and the next bind takes it again. No operator creates a thread, and no worker runs outside a bound model's operators.

Workers are parked rather than ended because ending them made the program's own exit fault: a thread of this runtime that has run procedures leaves work for Windows' thread-pool workers when it ends, and a program that ends soon after races them. With fault dialogs switched off, as a test harness does, 89 of 600 exits of a twelve-line program with no model in it reported an access violation; with the threads blocked instead, none did.

Two settings can lower the count; nothing raises it past the allowance:

```
--threads N             compile option, Windows only: at most N threads;
                        1 runs the model on the calling thread alone
PmModelSetThreads(n)    run time: at most n; 0 = every allowed processor
PmOnnxSetThreads(n)     the fixed-shape API's name for the same setting
```

Both procedures return the count now in use. On a bound model the workers are rebuilt at once; before binding the setting waits for the bind.

The thread count never changes an output bit. Each operator is cut into tasks by output - elements, rows, channels, positions - and every output value is computed by one task with exactly the arithmetic and the summation order it has on one thread. No sum is ever divided between threads. So 1 thread and 32 give the same bytes, and the Windows program still agrees with the Pi 4 and the Picos wherever it did before.

An operator smaller than its measured break-even stays on the calling thread; handing a hundred additions to 31 other threads costs more than doing them.

> **Note.** What is not split, today: the operators whose kernels are in

`tensor_ops.pmi` - the pools, the quantized products and convolutions, GRU and RNN, and the rest of the operator set added in 2026 - run on the calling thread. That file is one source for every target and keeps its parameters and scratch in shared globals, so two threads cannot run it at once without a change to it that every target would see. The bare-metal targets keep their own core use, chapter 6.

> **Note.** A program may end without closing its model - on an error path, say.

Bind returns only once every worker has started and is blocked, and a worker between operators is always blocked, so the exit is clean. Close the model anyway when you are done with it: that is what frees its memory and its workers' buffers.

The resident lifecycle, stated once

Bind once. Submit inputs. Read outputs. Release.

- **Bind** validates the pack, decodes reduced-storage weights into resident FP32 memory, takes the Windows worker threads, and runs the model's constant-preparation stage. This is where the CRC and the identity check are paid for. - **Submit** fills the input buffers - or, on the runtime-dimension path, declares this request's shapes and then fills them. - **Execute** runs the emitted schedule. Outputs are valid until the next execution or reset; they are borrowed, not given. - **Reset** frees the request's temporaries and keeps the model. This is the step that makes the next request cheap, and it is the step people forget. - **Close** or **unbind** releases everything and, on Windows, parks the worker threads. On the portable targets your buffers are still yours and still need freeing.

> **Note.** The generated globals are ONE model instance. Serialize your calls.

There is no per-instance handle, so two threads in the same model are two threads in the same state. Nothing in the runtime will stop you and the result will not be an error message.

Watching a long run, and stopping it

A resident model can run for a long time - minutes, on the speech model in chapter 7 - and a program with a watchdog cannot simply disappear for that long. So the reusable runtimes expose a node-boundary callback:

```
DSetProgressCallback(callback)    ; callback(phase, node) -> nonzero to continue
                                  ; 0 disables it; none installed by default

PMD_PROGRESS_BIND_BEFORE  1   node -1
PMD_PROGRESS_BIND_AFTER   2   node -1
PMD_PROGRESS_NODE_BEFORE  3   node = the generated node index
PMD_PROGRESS_NODE_AFTER   4   node = the generated node index
```

Returning zero, or setting `DCancel`, asks for cancellation: the callback unwinds normally and the operation then takes its ordinary failure path, so you inspect `DError`, `DNode` and `DCancel` and do the normal reset.

> **Trap.** The callback runs synchronously on the inference thread and **must

not re-enter the model**. Every entry point rejects re-entry before it changes any state, records an error and sets `DCancel`. Keep the callback short. It is the right place to pet a watchdog or do required cache maintenance; it is the wrong place to do anything that might call back in.

And be honest with yourself about what it bounds. Progress points are node boundaries. They do not bound the time spent INSIDE one operator, one CRC pass or one allocation - and on a large convolution that is where most of the time is.

The five targets

One desktop, two 64-bit Arm boards with a payload contract, and two microcontrollers with a budget that says no.

The target is the destination, not the desk

The generator always runs on Windows x64. `--target` picks where the SOURCE is going: which dialect is written, which execution sources are copied out, which kernels are selected, which launch contract the emitted API assumes, and whether a capacity check applies.

A target ID that does not exist is refused by name, before anything is read:

```
> ... --compile output\twice.onnx --output out\t --target rpi5
ONNX COMPILER ERROR: unknown target rpi5
```

The five that do exist are listed by `--list-targets` in chapter 2.

Generating `pico2` source on a desktop does not need a Pico 2 attached and does not produce firmware. It produces source for a firmware you will build with a toolchain that is not in this repository.

Windows

The only target you can build with nothing beyond the compiler you already used in chapter 2. Emits `.pb`, uses native x64 SSE2/AVX kernels on every processor the process may use (chapter 5, "Threads on Windows"), and takes its weights from a file path.

```
| what you build it with  | the host language, x64, thread-safe + optimizer |
| you supply              | input data, output handling, the .pmw file      |
| initialization          | PmModelInitialize(weightPath) - it reads the pack |
| threads                 | every processor the process may use; --threads N or PmModelSetThreads(n) lower it
|
```

Use it as the correctness bench even when the destination is a board. It is the fastest place to find out that a graph does what you expected, and the answer transfers: the bare-metal kernels are held to bit identity against the same portable reference (chapter 9).

> **Note.** "SIMD" and "bit-identical" are not automatically compatible. Vector

paths that change floating-point summation order are documented as tolerance-based, not exact. Where a kernel claims bit identity it is because it was written to make that claim and proved against the scalar reference; where it does not, do not demand it.

Pi 4 and UNO Q

Both emit the same 64-bit Arm dialect, each under its own extension: `.pi4` for the Pi 4 and `.unoq` for the UNO Q (the PureMetal compiler takes the chip from the extension, and a UNO Q build refuses a `.pi4` library). They differ in how the

program is launched: the Pi 4 profile is entered as a bare-metal payload with caller-supplied memory; the UNO Q profile is a 64-bit UEFI application, and the loader, the weight loading and the device I/O belong to your application.

Both select the Advanced SIMD runtime and the 64-bit Arm math template. Neither is a companion-microcontroller sketch.

```
> ... --compile output\dense.onnx --output out\dense-pi4 --target pi4 --precision int8
PASS: generated ONNX source for pi4 (int8)
Source: ...\out/dense-pi4.pi4
Weights: ...\out/dense-pi4.pmw (196 B)
Quantized weights: 1 (per-output-channel signed INT8, dynamic per-row activation scales, INT32 accumulation)
Arena: 4.19 KiB including bounded inputs/outputs
Manifest: ...\out/dense-pi4.json
Resident model: bind weights/arena once, fill inputs and execute repeatedly, then unbind when finished.
```

The payload contract, in three lines of your Main

The emitted source says this at the top of the file, on these two targets, before anything else:

```
; BUILD THIS AS AN ANVIL PAYLOAD WITH --wants-services, and call
; PmOnnxAnvilEnter(x0) as the first statement of Main. Without both, every
; convolution runs on one core - the same bytes, several times slower.
```

A payload entered by the board monitor with the service flag set is handed a service table in `x0`. That table is how the model reaches the other cores. The emitted library defines the one call that consumes it:

```
; Call this FIRST in Main, with the value the monitor left in x0, when this
; payload was built --wants-services. Returns the core count; 1 when there
; is no parallel service. It never fails and never needs checking.
Procedure.i PmOnnxAnvilEnter(services.i)
  If PmSetAnvilServices(services) = 0 : ProcedureReturn 1 : EndIf
  ProcedureReturn PmParallelCores()
EndProcedure
```

Three properties of that call are worth stating plainly, because they are what make it safe to have at all.

It is safe not to call it, and safe to call it with zero. Either way every convolution takes the single-core path and produces **the same bytes**. A host application that has not been updated is slower and is never wrong. That is the fallback, and it is a real fallback rather than a hopeful one.

It does not go looking for the table by itself. There is no correct value for the library to guess, and guessing an address that is not a service table is a wild branch on a machine with no memory protection. So the host, which knows, passes it in.

It cannot fail. It returns the number of cores a kernel will be run on, and 1 when there is no service. There is nothing to check.

> **Trap.** The source DEFINING `PmOnnxAnvilEnter` is not the same as your

program CALLING it, and the difference is invisible: a payload that never calls it runs perfectly, produces byte-identical output, and is simply several times slower. That exact mistake was made on the bench - a host runner was built against an emitter that emits the entry, and never added the call, so it ran single-core and looked fine. If you expect parallel convolution, check the returned core count rather than assuming.

> **Trap.** Ask for the core count at the right moment. It is **not constant for

a payload's life**: a payload that asks before it has enabled its own memory management and caches can be told 1 and then get more later. Read it after your setup, not before.

Quantized convolution uses the cores too. It did not always. The convolution operator picks its kernel from the weight's element type before it reaches the dispatcher that asks for cores, and for a while the quantized side of that branch had no split at all - so an INT8 model ran every convolution on one core no matter what this call returned. It is the same shape of mistake as the trap above, one layer down, and it was equally invisible: one core returns the same bytes as four. Both precisions now take the same route, with one difference that is forced by the arithmetic - each batch of the input is quantized once, on the core you were entered on, before any partition starts, because every input position's scale needs every channel and that pass writes the buffer every partition then reads. Each partition then computes its own output channels in its own tile area of the model's working memory, so no two cores ever write the same bytes.

The four kilobytes of those tile areas are why an INT8 model's arena is 4 KiB larger on these two targets than on the others: the dense model above needs 4.19 KiB here and 256 bytes on a Pico.

> **Note.** The evidence that a split really ran is the service's dispatch

counter and the elapsed time, never the number `PmOnnxAnvilEnter` handed back. That number is a report; the kernels ask again, later, when they need an answer.

Runtime-dimension FP32 matrix multiplication uses independent row partitions when Anvil supplies multiple cores. The split starts at 32 rows, 16 columns and 65,536 multiply-add pairs. Each partition has its own packed weight panel (64 bytes per reduction element), allocated from the working arena before dispatch. Four-row boundaries preserve the existing vector tiles; the final partition owns the remaining rows. Each output retains the same sequential multiply/add order. A missing service, a refused dispatch or insufficient packing memory uses the existing single-core implementation. This does not allow concurrent requests to the generated model instance.

The Pi 4 proof on 2026-10-19 used Anvil build 328, Kitten nano 0.8 FP32, the Rosie voice, four requests and captured random inputs. Model execution fell from 66.534 to 57.413 seconds for 595,600 samples at 24 kHz. MatMul node time fell from 13.498 to 4.362 seconds; the complete waveform remained byte-identical. This is still slower than real time. Nine additional four-core matrix cases covered row and column tails, signed zeros, subnormals, infinities and NaNs against the serial packed kernel.

What the Pi 4 has actually done

This is a real workload on real silicon, and the conditions matter as much as the numbers. All of it is the full speech model of chapter 7, running resident, on a Pi 4 at approximately 1.5 GHz, timed with the board's own 54 MHz architected counter.

```
Request 2: "You are hearing a complete sentence produced on this small
computer."  -  69 tokens, 102,000 samples, 4.25 s of audio.

  2026-09-10, first complete run          639.3 s
  2026-09-11, vector trig + exact MatMul  147.3 s
  2026-09-15, register-blocked Conv tile   91.6 s
  2026-09-15, later the same evening       88.1 s
```

Inside that last figure, by operator family, read off the board's own per-node tick rows:

```
  Conv            70.45 s      the 4x16 register-blocked tile
  everything else 10.80 s
  MatMul           2.78 s
  LSTM             1.87 s
  ConvTranspose    1.80 s   <- untouched between runs: the control
  Gemm             0.38 s   <- untouched between runs: the control
```

> **Note.** The two control rows are the most useful numbers in that table.

ConvTranspose and Gemm were not changed, and they came back identical to the millisecond across runs. That is what says the clock, the tick rate and the attribution did not move - so the rows that DID change are the kernels and not the measurement. A speed table without controls is a story.

And the contract held throughout: the waveform digests for the first two requests are **bit-identical** across every one of those runs. Faster, and the same bytes.

The same request on all four cores, with the call above in the runner, took 34.6 s, its convolutions 17.1 s, with the same bytes out. Chapter 11 is a whole sentence run that way, with the runner and a recipe that reproduces its payload byte for byte.

> **Trap.** Do not read those numbers as a performance claim for your model.

They are one graph, one board, one clock, one set of texts. What they establish is that the kernels moved and the output did not.

Pico and Pico 2

Two microcontroller profiles, for small models. The weights are embedded in the generated source as a checked array rather than loaded from a file, so use the generated symbol and size - never a guessed flash address.

```
| target | SRAM   | reserved | flash | reserved |
| pico   | 264 KiB | 24 KiB  | 2 MiB |  96 KiB  |
| pico2  | 520 KiB | 40 KiB  | 4 MiB | 128 KiB  |
```

The reserve is the tool's allowance for runtime and stack. It is checked before anything is emitted, and the refusal does the arithmetic for you:

```
> ... --compile big.onnx --output out\big-pico --target pico
ONNX COMPILER ERROR: model does not fit target pico:
  SRAM required 805.25 KiB (arena 781.25 KiB + runtime/stack reserve 24.00 KiB), available 264.00 KiB, short
by 541.25 KiB
largest tensors:
  x [1,100000] = 390.62 KiB [activation]
  y [1,100000] = 390.62 KiB [activation]
Try the explicit INT8 option, reduce the bounded graph/model, or choose a larger target. Nothing was truncated
or emitted.
```

Every part of that is doing work. It names what was required and what was available, it shows the arithmetic, it lists the tensors that dominate so you know where to look, it suggests the next thing to try - and the last sentence tells you that nothing

partial was written. There is no truncated output to clean up.

> **Note.** If the model already uses INT8, the last line changes to say so

rather than suggesting an option you have taken. An error message that recommends something you already did is an error message nobody reads twice.

> **Trap. This check does not know about your firmware.** It knows the

model's arena and the model's pack. It does not know your peripheral buffers, your display framebuffer, your network stack, your stacks, or your application's own state. Passing the capacity check means the model fits the budget the tool was given, not that your finished image fits the part. Look at your linker map. Leave room.

> **Trap.** The full speech model is not a Pico model, with any precision. Its

weights alone are 324 MB and its text assets are tens of megabytes. The speech switches refuse those target combinations outright rather than emitting something that could never bind, and chapter 7 shows the exact refusal. Use the Pico profiles for models that are genuinely small.

One rule for all four bare-metal profiles

Generating source is not a build, not a boot test, and not numerical or timing proof. It says the graph was understood and the code was written. Everything downstream - the compile, the link, the image, the memory map, the numbers on your hardware - belongs to your application's integration and test process, and has to be established there.

Speech, end to end

The largest thing this compiler has been pointed at: one pinned model, four checked assets, two adapters, and an honest account of how slow it still is.

Speech is an application of the compiler, not a second compiler

Nothing in this chapter is a special path through the model compiler. The speech model is compiled exactly like the two-value `Add` in chapter 3: the same parser, the same validation, the same emitters, the same weight pack. It is a much bigger graph - 2,463 nodes, 3,032 tensors - and it needs a 324,621,440-byte pack, but it goes through the same door.

What is added on top is an **adapter**: the code that turns text into the token IDs the model wants, picks a voice, and hands back the audio samples that come out. That layer contains no graph semantics at all.

> **Trap.** **This repository does not contain the model, the voices or the

pronunciation data.** It contains the code that reads them, the code that builds the packed forms, and the code that checks them. You can build the compiler, run every arithmetic example, and generate speech source without any of those assets - but the speech demo will not speak until you supply compatible ones and keep their licences.

The model is pinned, and the pin is enforced

One exact export. Not "a Kokoro model"; that one.

```
Full Kokoro-82M ONNX, full-precision v1.0 export
325,532,232 bytes
SHA-256 8fbea51ea711f2af382e88c833d9e288c6dc82ce5e98421ea61c058ce21a34cb
```

Point the speech switches at anything else and they stop:

```
> ... --compile output\twice.onnx --output out\t --target pi4 --kokoro-text
ONNX COMPILER ERROR: The speech adapter requires the documented full Kokoro-82M model. Check its SHA-256
identity.
```

> **Note.** Why pin at all, when the compiler handles a general subset? Because

the adapter is not general. It knows this model's token vocabulary, its voice tensor dimensions, and the exact shape of its inputs. A different export with the same operator set would compile fine as a MODEL and be wrong as SPEECH, and that failure would arrive as bad audio rather than as an error.

The four assets, and their contracts

```
kokoro_us_english.pmg2p    12,613,954 B   base English pronunciation pack
kokoro_us_extra.pmg2p       5,397,245 B   supplementary pronunciation pack
```

```
<name>.pmvoice                522,368 B   one prepared voice
the pinned model          325,532,232 B   as above
```

The base pack is required. The supplementary pack is required by the Windows reader and optional-but-recommended in the portable adapter. Each voice is one file.

> **Trap.** Size is not identity. The loaders validate the format, the header,

the extents, the payload CRC-32 and the model identity recorded inside. You cannot rename an upstream raw voice `.bin` into `.pmvoice`, or a dictionary JSON into `.pmg2p`, and have it work - and the refusal you get will be a format refusal, not a hint that you were close.

Preparing a voice

An upstream raw voice is exactly 510 rows of 256 little-endian FP32 values - 522,240 bytes. The packer turns that into the checked form:

```
> ...\PureMetalOnnxCompilerCLI.exe --kokoro-pack-voice C:\Models\af_heart.bin --output
C:\Models\af_heart.pmvoice [--source-sha256 HEX]
> ...\PureMetalOnnxCompilerCLI.exe --kokoro-verify-voice C:\Models\af_heart.pmvoice
```

Packing rejects NaN and infinity in the data, validates the finished header, checksums and payload, records the pinned model's identity, and **refuses to overwrite an existing destination**. `--source-sha256` lets you assert the digest of the raw file you obtained from a trusted place, so a substituted download is caught before it is transformed.

Verification is a separate command because a file at rest is a different question from a file being written:

```
> ...\PureMetalOnnxCompilerCLI.exe --kokoro-verify-voice missing.pmvoice
VOICE PACK ERROR: PMVOICE file is missing or has the wrong length
```

> **Note.** Dimensions alone cannot prove a voice belongs to this model. 510 by

256 is a shape, not an identity. The packer records what it was told and checks what it can; it does not claim more.

Preparing the pronunciation packs

Two commands to build, two to verify, and every input is pinned by digest before a byte is parsed:

```
> ... --kokoro-pack-base-dictionary  us_gold.json --silver us_silver.json --output kokoro_us_english.pmg2p
> ... --kokoro-pack-extra-dictionary cmudict.dict --base kokoro_us_english.pmg2p --output
kokoro_us_extra.pmg2p
> ... --kokoro-verify-base-dictionary  kokoro_us_english.pmg2p
> ... --kokoro-verify-extra-dictionary kokoro_us_extra.pmg2p
```

The supplementary pack is built **against the finished base pack**, which it consults so that it only adds words the base cannot already answer. That is why the order matters and why `--base` is not optional.

Each pack is deterministic: the same pinned sources always produce the same bytes, down to the hash-slot probe order. Packing refuses a malformed lexicon line, a repeated word, a pronunciation outside the pinned vocabulary, a key longer than 63 bytes, and an existing destination - and every refusal names its numeric code and what to check. The completed header, the extents, the payload CRC-32, every hash slot and every pronunciation codepoint are validated before the output is published.

> **Trap. The pinned digests include the line endings.** The two Misaki

files are pinned with CRLF line endings, as a clone with `core.autocrlf=true` checks them out; downloaded raw they are LF and are refused. `cmudict.dict` is pinned as the LF file, so clone it with `core.autocrlf=false`. Chapter 11 gives the exact clone commands and the digests they produce.

> **Trap.** A pack's exact size, entry count and payload CRC are **pinned in

several places at once**: the board runtime's validator, the shipped verifier, and the loader used for checking. That is deliberate, and it has a consequence people meet the first time they want to add one word. A pack with one extra entry is refused by all of them. Adding a word is not a dictionary edit - it is a contract change that moves every pinned value and rebuilds the tools. The upside is that the pack cannot drift silently; the cost is that it cannot drift conveniently either.

> **Note.** A word that is not in either pack is not an error. The

grapheme-to-phoneme layer falls back to spelling it out letter by letter. That is audible, correct, and occasionally comic: before the product's own name was added to the supplementary pack, the board said it as "K O K O R O", and the sentence went from 87 tokens to 100 because thirteen of them were the machine spelling.

Windows: the reading window

```
> New-Item -ItemType Directory -Force output | Out-Null
> ... --compile 'C:\Models\kokoro.onnx' --output output\kokoro-reader --target windows --precision fp32 --
speech-ui
```

That emits a `.pb` you build yourself, as usual, thread-safe and optimized. Keep its `.pmw` and `.runtime/` with it. In the window you select a `.pmvoice` and the base `.pmg2p`, with `kokoro_us_extra.pmg2p` beside the base file. Voice discovery is local - it reads the directory you point at and does not fetch a catalogue from anywhere.

The reader chunks text the way a book is read, queues audio through a bounded producer/consumer queue, and gives you Pause, Resume and Stop.

`--speech-ui` is Windows only, and says so:

```
> ... --target pico2 --speech-ui
ONNX COMPILER ERROR: The speech-window adapter requires Windows. Remove --speech-ui when generating a model
library for another target.
```

Pi 4 and UNO Q: the resident text adapter

```
> ... --compile 'C:\Models\kokoro.onnx' --output output\kokoro-pi4 --target pi4 --precision fp32 --kokoro-text
```

Use `--target unoq` for the UEFI profile. And the switch is refused anywhere it does not belong:

```
> ... --target windows --kokoro-text
ONNX COMPILER ERROR: --kokoro-text requires pi4 or unoq; Windows uses --speech-ui. Full Kokoro cannot fit Pico
RAM.
```

The adapter deliberately contains no filesystem, no audio driver, no allocation service, no compiler invocation and no board addresses. It is text in, tensor IDs out, samples back. Everything else is the application's.

```
PmKokoroTextBind(*pack,packBytes, *extra,extraBytes, *voice,voiceBytes)
PmKokoroTextSetVoice(*voice,voiceBytes)
PmKokoroTextPrepare(*text,textBytes,speed)   ; -> token count, or 0
PmKokoroTextRender(*text,textBytes,speed)    ; -> output tensor ID
PmKokoroTextClose()
```

A resident runner, in outline

The order is the whole design, so here it is as one shape. Every line is a place something can fail, and every one of them is checked in a real runner.

```
Main(x0):
  PmOnnxAnvilEnter(x0)                  ; first statement - chapter 6
  set up MMU and caches                 ; the application's job, not the model's
  PmModelBindMemory(weights, 324621440, arena, arenaBytes)
      ; validates the pack header, identity and CRC-32
      ; runs constant preparation. Minutes of setup happen ONCE, here.
  PmKokoroTextBind(basePack, baseBytes, extraPack, extraBytes, voice, voiceBytes)
      ; assets stay CALLER-OWNED and immutable until close or rebind

  for each sentence:
      PmModelResetRequest()
      id = PmKokoroTextRender(text, textBytes, 1.0)
      if id = 0: read PmKokoroG2pError / PmKokoroG2pErrorAt / DError and stop
      copy the samples somewhere of your own       ; they are BORROWED
      ; ... play them, or write them, or hash them

  PmKokoroTextClose()                   ; does NOT free your assets
  PmModelClose()
```

Trap. The returned audio is **borrowed**. It is mono FP32 PCM at 24 kHz,

valid until the next prepare, the next render, a model reset or the close. Copy it before you submit more text. Keeping the pointer and submitting the next sentence is the mistake, and it does not announce itself - the buffer is still readable, it just holds something else.

Two more limits worth knowing before you design around it: one serialized request at a time, and a text request is capped at 4,096 UTF-8 bytes plus the supported phoneme sequence length. Book-sized scheduling is yours.

What performance to expect today

Measured, on a Pi 4 at approximately 1.5 GHz, timed by the board's 54 MHz architected counter, FP32, resident model, single core unless stated.

```
"The sky is blue, and the sun is shining."
    41 tokens, 68,400 samples, 2.85 s of audio
    2026-09-10  433.6 s      about 152x slower than real time
    2026-09-15   57.6 s

"You are hearing a complete sentence produced on this small computer."
    69 tokens, 102,000 samples, 4.25 s of audio
    2026-09-10  639.3 s
    2026-09-15   88.1 s
```

> **Trap.** **This is not usable interactive speech, and the book will not

pretend otherwise.** A 4.25-second sentence taking 88 seconds is roughly 21 times slower than real time. It is a correctness checkpoint on the way somewhere, and the honest description of where it is today.

Where the remaining time goes, from the board's own per-node tick rows for that 88.1-second request: convolution 70.5 s, everything else 10.8 s, matrix multiply 2.8 s, the LSTM 1.9 s, transposed convolution 1.8 s, general matrix multiply 0.4 s. Convolution is 80% of it, and the measured cost is 0.642 ns per multiply-accumulate over 109.67 GMAC on one core.

> **Note.** Every one of those speedups was required to leave the output

unchanged, and did. The waveform digests for both sentences are bit-identical across the whole sequence of kernel changes. That is the contract chapter 9 describes, tested at model scale rather than in a fixture: a kernel that is four times faster and produces one different bit has not passed.

The numbers above are the single-core FP32 ones. Two things have been measured since, each in its own chapter: running the convolutions on all four cores through the payload service of chapter 6 - chapter 11 has a 295-token sentence rendered that way in 158.2 s, with the runner and the recipe that reproduce it - and INT8, which chapter 4 shows running this model at half the speed of FP32 on the same four cores.

Accuracy, stated the same way

Against one frozen reference session of the pinned model, comparing every final sample with no alignment and no trimming:

```
limits:   max absolute error 0.1    RMSE 0.01    correlation 0.99

sentence 1   0.0963 / 0.00429 / 0.99800     pass
sentence 2   0.2119 / 0.00612 / 0.99638     FAILS on max absolute error
```

That second row is recorded as a failure, because it is one. It is not widened, explained away, or re-described. What is known about it is in chapter 8.

Pico is not a destination for this

The Pico profiles exist for small models. The full model's weights are 324 MB and its text assets are tens of megabytes; no storage precision changes that by two orders of magnitude. The speech switches reject those target combinations outright rather than offering a demo that could never bind.

How it is proved

An independent oracle, a bit-identity contract where one is possible, and a plain statement of where the proofs live and why it is not here.

The oracle

To know whether a generated model is right you need an answer that did not come from the thing being tested. The reference used throughout is an independent CPU **ONNX Runtime float32** session over the same model and the same inputs, its outputs frozen to disk with their digests recorded beside them.

That engine is never linked into anything generated. There is an optional bridge in the compiler for host reference and tracing work - pinned to a specific C API version, needing a library you supply separately - and generated inference does not call it, does not know it exists, and does not require it to be present. It is a measuring instrument on the desk.

```
--trace-model    model.onnx output.onnx
--trace-shapes   model.onnx --ort-dll path --trace-input NAME=file.npy ...
--lower          model.onnx --ort-dll path --trace-input NAME=file.npy ...
--ort-version    [onnxruntime library]
--npy-inspect    tensor.npy
```

> **Note.** The NPY reader is not a Python dependency. The format is parsed

directly, with unsupported layouts refused rather than guessed at. It exists because reference tensors are commonly stored that way.

Bit identity, where it can be claimed

Most numerical work is graded with tolerances. A lot of it here is not, and the difference is deliberate.

When a kernel is written to replace another - a vector kernel replacing a scalar one, a blocked tile replacing a simple loop - the contract is that it produces **exactly the same bits**, not a close answer. The kernel headers say so in the source:

```
; NUMERICAL CONTRACT - this kernel IS bit-identical to the scalar body in
; PmFastBinary, and unlike the reductions above it is allowed to claim that.
; Each output element is one FADD/FSUB/FMUL/FDIV of the same two binary32
; operands in the same order, so it is the same single rounding; nothing is
; reassociated, contracted into an FMLA, or evaluated in a wider type.
```

And where a kernel CANNOT claim it, the source says that too, in the same plain way:

```
; Vector reduction and FMLA intentionally do not promise bit identity with
; the scalar FP32 reference. Floating-point addition is non-associative and
; FMLA rounds once. The board gate therefore retains exact scalar checks and
; gives optimized runs an explicit numerical contract.
```

Trap. A tolerance is the right instrument for comparing two independent

implementations of the same mathematics. It is the WRONG instrument for checking that an optimization did not change anything, because a tolerance will happily accept a kernel that has quietly started computing something slightly different, and the difference will grow the next time somebody optimizes on top of it. Where the answer should be identical, test for identical.

That contract is what the whole speech sequence in chapter 7 rests on. Across a run of kernel replacements that took one sentence from 639 seconds to 88, the two reference waveforms came back **byte for byte unchanged** every time. That is a stronger statement than any fixture could make, because it is the contract holding at model scale, through 2,463 nodes, with every intermediate a chance to differ.

What the published checks establish, precisely

The publication checks were run on Windows x64 with the native assembly backend: no Python, no inference-engine library for the demo, no board.

```
build helper and both entry points                          PASS
CLI bounded-wire self-test and target listing               PASS
native demo-model creation                                  PASS
generated Windows Add model, three changing-length requests PASS
dense INT8 generation for all five profiles                 PASS
internal Windows dynamic-runtime diagnostics                PASS
internal Windows SIMD diagnostics                           PASS
```

Read those as what they say. "Five-target emission" means five source closures were written and checked, not that five boards ran anything. The Add model check establishes repeated resident execution, correct values, request cleanup and final release - on one tiny graph.

The 64-bit Arm work has its own record. The native `Exp` implementation was compared against the retained software one across a 791-input differential corpus covering both dispatch rails, special bit patterns and reduction boundaries: all 723 admitted and 68 fallback cases agree **bit for bit**, and five structural negative controls reject changes to the fallback, the rails, the rounding, the polynomial and the no-fusion contract. A separate high-precision oracle measured a worst error of 1.167302 ULP on that corpus - which is a sample measurement, and is described as one, not as a bound.

The same 791-input probe then passed on Pi 4 silicon, with all three raw result arrays read back and compared.

Note. The negative controls are the part to notice. A test suite that only

ever passes is a suite nobody has checked. Deliberately damaging the thing under test and confirming the gate catches it is what makes a green run mean something. Several of these gates carry named mutants for exactly that reason.

The node-test score

The checks above prove particular things deeply. They do not say how much of ONNX the compiler handles. For that there is one number, measured against a corpus nobody here wrote: the official ONNX backend node tests. Each case is one small model with its inputs and its expected outputs.

A developer harness in `tests/node_suite/` compiles every case for the Windows target, builds one generic driver around the emitted source, runs it with the case's inputs, and compares every output: shape and element type must match, integers and booleans exactly, floats with the tolerance the onnx test loader itself assigns (rtol 1e-3, atol 1e-7). Every case ends in exactly one outcome.

```
corpus     onnx 1.22.0, 1,765 node cases
           sha256 4955443be1453848b40f58dd92d8e709907cb73a0141cf17d0a9346cb863ce6b
scored     964 cases at ai.onnx opset 20 or lower
compiler   commit 6bd59b0, target windows

PASS           95
FAIL_NUMERIC   40
REFUSED       829
FAIL_SHAPE, FAIL_DTYPE, BUILD_ERROR, RUN_ERROR, HARNESS_ERROR   0

PASS 95 of 135 attempted (70.4%); PASS 95 of 964 scored (9.9%)
```

Read the two ratios separately. **Attempted** is the cases the compiler accepted, and there 40 answers were wrong: eight defects in operators it claims, listed in chapter 4 and filed before anything was changed. A ninth, Gemm without a bias producing source that did not build, was fixed at its source the same day; before that fix the score was 94 of 135. **Scored** is the whole corpus, and there the 829 refusals are the size of the job ahead: 471 name an operator that is not implemented, 157 are the runtime-dimension path accepting only opset 20, 102 come from shape inference inside function-expanded tests, 90 are element types, and a handful are old opsets and rank forms.

That table is the measurement at that commit and stays as it was measured. Since then the eight defects it found have been fixed (forum topics 848 and 850 to 856) and more operators have landed, and the same corpus and harness at commit `8b7d9df` give **PASS 166 of 166 attempted (100.0%); PASS 166 of 964 scored (17.2%)**: no accepted case computes a wrong answer, 798 cases are refused with a sentence, and no case that passed at `6bd59b0` fails.

The per-operator table, every refusal grouped by operator and every failure with its detail, for the latest measurement, are committed beside the harness, in `tests/node_suite/results/`.

```
py -3.12 -m pip install onnx==1.22.0
py -3.12 tests\node_suite\node_suite.py --pbcompiler "<install>\Compilers\pbcompilerc.exe" --from-commit HEAD
py -3.12 tests\node_suite\node_suite.py --pbcompiler "<install>\Compilers\pbcompilerc.exe" --from-commit HEAD
--self-check
```

`--from-commit` builds the compiler from that commit, so a half-finished edit in the working tree is never what gets measured. The harness refuses to score a corpus whose onnx version or hash differs from its pin, and refuses to write a score while any case ends in HARNESS_ERROR. A full run takes about 35 seconds on 16 parallel jobs.

> **Note.** The harness is a developer check, and it and the targeted cases beside

it are the only Python in this repository. Building the compiler and using it never need them, and the statement that no Python is required is about exactly those two things.

Forms the node tests do not reach

Most of the official node tests for Cast, Shape, Conv, ConvTranspose and LSTM are generated at opset 22 or 25, above the harness's ceiling, and none of them writes an attribute out at its default the way an exporter does. So `tests/node_suite/targeted_defaults.py` builds its own opset-20 cases: the default-valued attributes of those operators and of LayerNormalization, Resize, ScatterND and ReduceSum, the non-default forms that are implemented (auto_pad SAME, Shape slices, ScatterND reductions), and the forms that must be refused. Each is built twice - extents declared, and one extent symbolic - so both emitters answer for it. The expected outputs come from the ONNX reference evaluator and are held against ONNX Runtime; where the reference itself departs from the specification text (its Conv auto_pad, its ScatterND max and min) the case says so and feeds it the specification's arithmetic instead.

```
py -3.12 tests\node_suite\targeted_defaults.py --pbcompiler "<install>\Compilers\pbcompilerc.exe" --cli
bin\PureMetalOnnxCompilerCLI.exe
targeted: 148 of 148 as expected
```

A refusal case passes only if the compiler refuses and the sentence names the operator. The same 148 builds on the commit before the attribute work gave 45: 72 refused because an attribute was merely present, 25 wrong values on the fixed-shape path, five forms accepted that should have been refused, and one compiler crash.

> **Trap.** A harness that reports PASS for everything is worse than none, so

this one has to prove it can say no. `--self-check` nudges one expected element to ten times the tolerance and requires FAIL_NUMERIC at that index; swaps two same-shape expected outputs and requires a failure while the unswapped control passes; compiles a model using an operator that is not implemented and requires REFUSED naming it; and runs a known-good case twice from nothing and requires identical bytes. All six checks passed on `6bd59b0`.

Control flow and sequences: forms the node tests do not reach

The official node tests for `If`, `Loop` and the sequence operators are few, and the `SplitToSequence` ones are published at opset 24. `tests/node_suite/targeted_control.py` re-imports those at opset 20 and scores them against their official outputs, and adds the forms a real model uses: branches of different rank, a while loop, zero trips, carried values that trade places, a loop inside a loop with an `If` reading values two scopes out, a sequence grown over 300 trips, and Kitten TTS nano's own nodes cut out unchanged. Expected outputs come from ONNX Runtime, checked against the ONNX reference evaluator where it runs the model; where the two disagree the case is thrown out rather than scored.

```
py -3.12 tests\node_suite\targeted_control.py --pbcompiler "<install>\Compilers\pbcompilerc.exe" --cli
bin\PureMetalOnnxCompilerCLI.exe --kitten kitten_tts_nano_v0_8.onnx
control: 34 of 34 as expected
```

`tests/node_suite/pi4_control_gate.py` takes the same case folders, builds each for the Pi 4 around a small returning payload that fills the inputs (sequences included) and serialises the outputs, and runs the image in an independent A64 instruction interpreter. It found one thing a Windows run cannot: a result written four bytes at a time after three BOOL bytes is an alignment fault on the Pi 4 with its MMU off. The sequence block keeps every field on a 4-byte boundary and every element on an 8-byte one for that reason.

Where the failure is, and why it is still recorded as a failure

Chapter 7 ends with a passing sentence and a failing one. The failing one has been investigated in detail, and the investigation did not produce a defect. It is worth summarizing, because it is a good example of how a number can be a real failure and still not be a bug.

No operator has been found defective. The divergence starts in the text encoder at about 6e-5 - inside the reference engine's own fusion and threading noise - and is amplified roughly 3400 times by a chain of cumulative-sum, multiply and resize operations. Where the sine is taken, 97% of its arguments exceed 12,867 radians, and at that magnitude one representable float32 step moves the sine by as much as 0.0156. The phase is ill-conditioned, and both float32 answers are further from the exact sum than they are from each other.

Measured against a double-precision evaluation of the same graph, the board is **nearer the mathematical answer than the reference is**, on every metric of both sentences. Ten configurations of the reference engine span 0.0973 among themselves while the limit is 0.1 - so the reference is a point in a cloud rather than a fixed line.

> **Trap.** None of that makes the failing number a pass, and it is not treated

as one. "We are closer to the true answer than the yardstick is" is evidence about the contract, not compliance with it. The result stays recorded as a failure with its evidence beside it until the acceptance contract itself is reviewed and settled. Changing a limit because a measurement went the wrong way is the one move that would make every earlier passing result meaningless.

The random operators: the same numbers on every target

The ONNX specification does not say how random values are generated, and a reference runtime does not reproduce its own. So the ONNX node tests for these operators could only ever check shape and element type - and the onnx 1.22.0 corpus has none at opset 20 or lower at all (its three Bernoulli expansions that use `RandomUniformLike` are opset 22). The contract below is this compiler's, it is exact, and it is what the gates hold the code to.

The generator is Threefry-2x32 with 20 rounds, the counter-based function of the Random123 library and of JAX's `threefry_2x32`. It uses only 32-bit addition, rotation and exclusive or, so it needs no multiplier and no 64-bit integer, and the same source is exact where the native integer is 32 or 64 bits.

The streams. Each node has its own key, derived from a seed and the node's index (its 0-based position in the graph):

```
node key  = Threefry(key = (S, 0), counter = (node index, d))
            S = IEEE-754 bits of the node's FLOAT seed attribute, d = 1, when it has one
            S = the model seed (32 bits, 0 until set),            d = 0, when it does not
element i = Threefry(key = node key, counter = (i, request number))  ->  words w0, w1
```

A node never consumes another node's draws, so adding a random node leaves every other node's values alone - unless it is inserted before them, which changes their index and so their key.

The transforms. Every floating step is one binary32 addition, subtraction, multiplication or division with a correctly rounded result, or an exact conversion of an integer below 2^25. No maths library is called: the targets' own square root, logarithm and cosine differ by a unit in the last place, and one ULP is enough to break a bit-exact contract.

```
uniform   out = low + (high - low) * u,      u = (w0 >> 8) * 2^-24 in [0, 1)
normal    out = mean + scale * (r * c)       Box-Muller, cosine branch
```

```
          u1 = ((w0 >> 8) + 1) * 2^-24 in (0, 1]
          r  = sqrt(-2 ln u1): ln from an exact power-of-two split and the
               atanh series; sqrt by a fixed four-step Newton sequence
          c  = cos(2 pi u2): the top three bits of w1 pick an octant, the
               next 21 the angle inside it, cosine or sine from Taylor chains
          |r * c| <= sqrt(48 ln 2) = 5.77
Bernoulli    1 where u < p, else 0            u as for uniform
Multinomial  the first class whose running sum of exp(x - max), added in
             class order, exceeds u * total (exp from tensor_ops.pmi);
             else the last class with a positive weight, else class 0
```

The uniform output lies in [low, high]: with the default bounds it is below 1 (at most 1 - 2^-24), and for wide bounds binary32 rounding can return `high` itself for the largest `u`. The normal output is truncated at 5.77 standard deviations, where the true distribution has 8 in a billion of its mass.

What is proven, by two gates in the private tree:

- The Python reference reproduces the three published Threefry-2x32-20 known answers, and two independent Python implementations of the transforms agree bit for bit. - The generator source, built for Windows, writes the known answers, 832 crafted words covering every octant and every edge of the logarithm, six streams, and two 4,000,000-element streams, all bit-identical to the reference. Built for the Pi 4 and run in the A64 interpreter, and built for the Pico (software binary32 on a Cortex-M0+) and the Pico 2 (Cortex-M33) and run in the ARM emulator, it writes the same known answers, the same 832 words and the same six streams, bit for bit. The streams' SHA-256 digests are printed side by side. No board was used. - Over the 4,000,000-element streams: the normal mean and variance are within five standard errors of 0 and 1, the uniform mean and variance within five of 1/2 and 1/12, both Kolmogorov-Smirnov distances are below twice the 5% critical value, every uniform value is in range, and changing the model seed, the request number, the node index or the seed source leaves fewer than 0.1% of values equal and no correlation beyond 5/sqrt(N). - Six mutants of a copy of the generator - a rotation, a key injection, a series coefficient, the octant fold, the request word, the Newton count - are each rejected. - Through the compiler: eight node-test-style cases covering every claimed form on both paths pass the node-test harness's own run and are then compared bit for bit; nine unclaimed forms are refused on three targets with sentences naming the operator, attribute and value; both APIs run a sequence of requests with seed changes and a rebind, bit-identical to the reference; `--random-inputs` passes supplied noise through exactly and refuses a wrong shape; fixed-shape and runtime-dimension Pi 4 builds run in the A64 interpreter bit-identical; Pico, Pico 2 and UNO Q sources are emitted and built in both modes; and the two random nodes of the Kitten speech model, extracted with their consumers and attributes untouched, validate, emit for Windows and the Pi 4 in both modes, and run on Windows in both.

> **Note.** The generator is not vectorised. It is cheap beside the operators

around it in the models that use it, so there is no SIMD or NEON body and no second implementation for the contract to hold between.

InstanceNormalization: a reference that can be vectorised exactly

The specification says `y = scale * (x - mean) / sqrt(variance + epsilon) + B` and says nothing about the order the mean and variance are summed in. A plain running sum cannot be computed four elements at a time with the same bits, so the reference defines an order that can: four running sums, element k of a plane going to sum k mod 4, combined as (s0 + s1) + (s2 + s3). Every other step is one binary32 operation in a stated order, and the square root is the correctly rounded one. The SSE body on Windows and the Advanced SIMD body on the Pi 4 then do exactly those operations a lane at a time, and the contract for both is identity, not a tolerance:

```
PASS SSE InstanceNormalization: 0 of 14738 elements differ from the scalar reference; a one-unit epsilon
change moved 410
PASS NEON vs scalar InstanceNormalization: 0 of 5166 elements differ
PASS mutation: NOP at pmtensorinstancenormlanesneon+124 gives 1710 mismatching elements
```

The planes those numbers come from are built to be hostile - every width from 1 to 41, signed zeros, infinities, subnormals, the largest finite values and NaNs with payloads - and the mutation line is the negative control: one accumulating instruction removed from the vector body, and the comparison notices.

The node tests for InstanceNormalization, TopK, Pad and Identity are all published above opset 20, where their later definitions only add element types, so `tests/node_suite/targeted_norm_small.py` re-imports them at opset 20 and scores them against the official outputs unchanged, beside its own cases for the forms the Kitten voice uses:

```
py -3.12 tests\node_suite\targeted_norm_small.py --pbcompiler "<install>\Compilers\pbcompilerc.exe" --cli
bin\PureMetalOnnxCompilerCLI.exe
norm_small: 132 of 132 as expected
```

INT8: one definition, the same bits on every target

Chapter 4 states the INT8 arithmetic step by step. Every step is an exact integer operation or one binary32 operation, rounded to nearest-even, in a stated order - a row maximum taken on bit patterns, one division each for the scale and its reciprocal, one multiply and a round-half-even per value, exact integer sums, and per tap and per 512-element chunk one conversion, one multiply and one add. Nothing is fused and nothing is reassociated. That makes identity, not a tolerance, the contract between the Windows kernels (with AVX2 and without), the 64-bit Arm Advanced SIMD kernels of the Pi 4 and UNO Q, and the Cortex-M kernels of the Pico and Pico 2, whose binary32 arithmetic is done in software on the Pico.

What is proven, by gates in the private tree, with no board:

- The definition is also written in numpy, from the description rather than from either implementation. On 51 cases - MatMul, Gemm in all four transpose forms with its bias, Conv1D with padding, strides up to 6, dilation and groups, Conv2D, the recurrent layer's gate rows - covering narrow and wide weights, reductions that cross the 512-element chunk, rows of zeros, rows below 1e-30, subnormal rows and values that land exactly halfway between two integers, the Windows kernels with AVX2, the same kernels without it, and numpy give the same bits. The Pi 4 build (run as A64 code under an instruction emulator), the Pico build and the Pico 2 build (run as Thumb code) give those bits too, and the working memory they are handed is filled with NaN first, so a kernel that read memory it had not written would show it. - A NaN or an infinity in the activations is refused on every target with the output left unwritten, and working memory one byte short of what an operator needs is refused rather than overrun. - A recurrent layer's output passes through its sigmoid and tanh, which are each target's own FP32 functions, so the layer is held to its gate rows exactly (read back out of working memory on the embedded targets) and to its outputs within 1e-6 of the largest value for one step. Over a sequence the requantization of the hidden state can round a value the other way, so sequences are held to 1e-5 - measured 1.5e-7, against 2.5e-7 for the same layer with its weights left in FP32. - Seventeen runtime mutants - rounding half away from zero, truncation in the vector quantizer, a 1024-element wide chunk, the scale distributed over the two planes, uncleared padding scales, rows below 1e-30 kept, NaN not refused, a vector step that drops a channel, a multiply where the epilogue divides, and more - are each rebuilt and each caught. - A small model with every INT8 operator form, compiled by this tool for both model interfaces and built for Windows, the Pi 4, the Pico and the Pico 2, gives the same bits from the generated programs end to end. - Nine nodes taken out of a real request of the speech model - the text encoder's wide convolution, the ALBERT feed-forward MatMul, a stride-6 wide convolution of the waveform generator, 7.1 billion multiply- accumulates of one generator convolution, and more - were fed their real inputs on the Pi 4 build (whole) and on the Pico and Pico 2 builds (their first output channels or rows): every output

bit matches the Windows run. - The Pi 4's four-core convolution: each partition's worker, run on a hand-built partition block, writes only its own channels (the others are poisoned first) and the pieces reassemble to the whole, bit for bit, in one to four pieces; the vector tile runs and no FP32 tile does once an INT8 weight is bound.

What was measured, and on what. The emulators count instructions; the Pico one also models cycles. On real speech-model convolutions the Pi 4 build issues 0.38-0.41 A64 instructions per multiply-accumulate on the narrow layers against 0.63-0.68 for the FP32 kernel, and 0.88 against 0.82 on the wide ones - 1.58 times fewer over four representative layers. On the Pico and Pico 2, in the cycle model, the gate model's INT8 request takes a third of the FP32 one's time. **Neither is a measurement on silicon, and the Pi 4 speed of this scheme is still to be measured on the board.**

The operator set: one kernel source, the same bits everywhere

The operators added in September 2026 (chapter 4 lists them) have one kernel file, `runtime/tensor_ops.pmi`, written in the subset of the language both dialects accept, and included unchanged by the Windows program and by every bare-metal one. One source is not yet one answer: the targets' floating-point hardware and libraries differ. So the file keeps three rules.

One binary32 operation per statement. The host compiler evaluates a float expression on the x87 stack at extended precision; a statement with one operation rounds once on the way back to memory and gives the correctly rounded result, which is what the Arm cores and the Pico's software floats give. The functions the kernels need - exp, expm1, log, log1p, tanh, erf, sqrt, tan, asin, acos and the hyperbolic functions - are written into the file as fixed sequences of such statements (erf follows fdlibm's s_erff.c; sqrt is the integer square root of the significand, correctly rounded; tan reduces its argument by pi/2 in integer arithmetic against 288 bits of 2/pi, so a large argument keeps its quadrant), not taken from each target's library. The one step the reference takes in binary64 - the quantized operators' INT32 accumulator times a binary32 multiplier, plus a zero point, rounded half to even - is reproduced exactly in integer arithmetic: the product formed in 12-bit digits, rounded to 53 bits as binary64 rounds it, and the sum's own rounding applied where it can move the result, near a half.

Decide NaN on the bits. The host compiler's float comparisons do not order a NaN the way IEEE 754 does before version 6.41, and the answer depends on the operand order and on literals: with v a NaN, `a > b`, `a >= b` and `a = b` are true, `a <> b` is false, and `v < 0.0` and `v <= 0.0` are true while `v > 10.0` is false. 6.41 answers as IEEE 754 does. The host compiler also adds -0 and -0 to +0. Every kernel therefore decides NaN on the bits (`PmOpIsNan`, `PmTensorIsNan`) wherever the answer depends on it - compiled only for an older compiler where it merely corrects the comparison (`#PMO_HOST_NAN_BUG`, set from `#PB_Compiler_Version`), always where ONNX itself asks for it - adds through a helper where a negative zero can arise, and writes every NaN it produces as one quiet NaN, `$7FC00000`, because the x87 and the Arm cores make different NaNs for an invalid operation such as infinity times zero.

A definition to be held to. `ops_scheme.py` in the compiler repository's `tools/onnx/tests/Diagnostics` writes every kernel out in numpy one binary32 operation per line. `ops_kernel_check.py` runs 604 cases - the math functions over tables of special values and branch boundaries, every operator's kernel, and the awkward cases - on Windows and, in the unicorn instruction emulator, on the Pi 4, Pico and Pico 2 builds, and once more on Windows with the branch an older host compiler builds (`windows-hostbug`, `#PMO_HOST_NAN_BUG` forced to 1), and requires every output to equal the definition bit for bit; with `--mutants` it plants 72 defects and requires each to be caught:

```
py -3.12 ops_kernel_check.py --pbcompiler <pbcompilerc.exe> --compiler PureMetalForge.exe --runtime <ONNX
repository>\runtime --mutants
windows 604 of 604 cases as the definition
windows-hostbug 604 of 604 cases as the definition
pi4     604 of 604 cases as the definition
pico    604 of 604 cases as the definition
```

```
pico2   604 of 604 cases as the definition
mutants: 72 of 72 killed
```

That says the kernels compute what they were written to compute. Whether that is what ONNX specifies is the node tests' question: `tests/node_suite/targeted_ops.py` re-imports the official cases these operators have above opset 20 at opset 20, adds its own forms for both emitters, and scores everything against the ONNX reference evaluator, cross-checked with ONNX Runtime. Where the reference evaluator cannot run a form (RNN and GRU with sequence lengths or both directions, NonMaxSuppression without its thresholds) ONNX Runtime decides alone, and so it does for GridSample's cubic mode with border padding, where the reference evaluator clamps the coordinate as well as each tap and ONNX Runtime and PyTorch clamp only the taps; where it lacks one (GroupNormalization-18, Upsample-7) or answers otherwise than the operator's definition (LpNormalization, LRN, bilinear Upsample) a numpy definition decides, still cross-checked with ONNX Runtime wherever ONNX Runtime runs the form; RoiAlign at opset 10, to which it applies opset 16's default, is run at 16 with the old mode spelled out. `ops_targets_gate.py`, beside the kernel check, then compiles the same cases for the Pi 4, Pico and Pico 2, runs the generated programs in the emulator, and requires their outputs to pass the same comparison and to equal the Windows program's bits. The runtime-dimension support files, which every such program carries, have a gate of their own: `runtime_mutants.py` plants a defect in a copy of them, builds the compiler from that copy, and requires the cases that cover the procedure to fail - on Windows for the Windows runtime, and in the emulator on the Pi 4, Pico and Pico 2 for the portable one.

Threads on Windows: the same bits at every count

The Windows runtime splits an operator across its workers only by output (chapter 5), and one gate holds it to that. `mt_split_gate.py`, beside the kernel check, builds a program against the runtime folder that runs every operator the runtime splits - element-wise and row-wise kernels, FP32 and INT8 products, every convolution form, the recurrent steps, the transform, data movement and reductions, the normalizations and the rest - at 1, 2, 3 and 8 threads, once with the natural split and once with every operator cut into its smallest tasks, on odd shapes so no two tasks are the same size. Every output's CRC must equal the one-thread run's; an operator that should have split and did not fails, and so does a tiny tensor that did. A second program narrows its own process's affinity mask and requires the detected processor count, the default number of threads and the processor every task ran on to follow it. A third ends a program 180 times - straight after binding, after one operator, after unbinding - launched with fault dialogs off, as the node-suite harness launches, and requires every exit to be the program's own. Seven planted defects must each turn it red:

```
py -3.12 mt_split_gate.py --pbcompiler <pbcompilerc.exe> --runtime <ONNX repository>\runtime --mutants
split: PASS
  CASES 144 x 8 configurations, 0 failures
affinity: PASS
  mask $1: 1 processors, default pool 1 threads
  mask $F: 4 processors, default pool 4 threads
  mask $5555: 8 processors, default pool 8 threads
  ...
exit: PASS
  exit: 180 of 180 ends were clean (bind, run, unbind; fault dialogs off)
mutant reduction-split: red  (a row's sum taken in two halves)
mutant racy-scratch: red     (every worker in one working buffer)
mutant no-join: red          (the run returns before its workers finish)
mutant ignore-mask: red      (the machine's count instead of the process's)
mutant no-start-wait: red    (bind returns before its workers have started)
mutant spin-on-language-memory: red  (workers spin on memory End releases)
```

```
mutant end-at-unbind: red    (unbind ends the workers instead of parking them)
mutants: 7 of 7 red
```

It says the split never moves a bit and never escapes the processors the process was given. It does not say anything about speed; chapter 4 has the measurement.

Where the gates live, and why not here

`tests/` is **not tracked by this repository**, with one exception. Its `.gitignore` excludes it under "build products and local model data are not source", and the validation page states that internal diagnostics are not part of this export. The exception is `tests/node_suite/`, the node-test harness above: it measures against a public corpus, so anyone with this checkout can reproduce its number, and it is tracked for that reason.

So the gates - the differential corpora, the mutation suites, the board fixtures, the oracle scripts - live in a private tooling tree, where they are tracked, pushed and backed up. That is the rule, and it comes from a failure worth naming: a gate written into an untracked directory beside the code it tests exists on exactly one disk, and survives exactly as long as that working directory. The code it proved goes on shipping; the proof evaporates. A test in an ignored folder is not a proof.

> **Note.** This is also why you cannot reproduce every number in this book from

this checkout. What you CAN reproduce from this checkout is every command in chapters 2 through 6, the whole arithmetic walkthrough, the generation of all five source closures, and the node-test score. What needs the private tooling is the differential corpora and the board fixtures; what needs hardware is anything measured on hardware.

The limits, stated together

- A **validated subset** of ONNX, not the specification. Unsupported operators, attributes, element types and shape forms are rejected on purpose. Each operator is accepted from its floor to its ceiling, opset 27 at most, on both paths. Eight accepted forms are known to compute wrong answers today; chapter 4 lists them. - Windows x64 is the verified host. Other host systems, other versions of the host language, and other backends are not certified by this export. - Generating bare-metal source establishes no downstream build, no bootability, no hardware accuracy and no timing. The measured Pi 4 results cover their named fixtures, not other boards or other models. - A successful generation is not a promise that arbitrary future request sizes will fit. Dynamic requests still need memory and can still fail. - INT8 is selective. FP16, BF16 and INT4 reduce storage and do not remove the resident decoded FP32 cost or the activation cost. - SIMD paths may change summation order. Do not demand bitwise equality where the execution contract calls for tolerances - and do demand it where the contract calls for identity. - Generated models use generated globals. Serialize your calls. - Random operators follow this compiler's specified generator, not any other runtime's; their values match a reference only through `--random-inputs`. Only FLOAT output is generated. - Format checks are not a security audit. Trust your model sources.

Before you trust your own model

Start in FP32 with inputs whose correct outputs you already have. Check shapes and element types. Read the manifest's memory accounting and compare it with what the target really has. Exercise the error paths. Repeat requests and watch live storage return to its baseline. Then compare reduced precision on representative data, yourself.

For bare metal, validate on the hardware you intend to use before making any correctness or performance claim. Nothing in this book substitutes for that, and nothing in it is meant to.

Contributing

Where the code is, what the numbers are allowed to do, and what it takes for a kernel to be believed.

The shape of the tree

```
Build.pb              builds the two tools. Not part of the generator.
src/                  the application entry points
  compiler/           parser, IR, validation, precision, emitters, window
runtime/              execution source embedded into the generator and
  math/               exported beside every generated model
examples/             model creation and a complete Windows consumer
docs/                 the reference pages, and this book under docs/guide/
```

The compiler modules divide along one seam that is worth learning before you change anything: **the parser and the IR own what a graph MEANS; a target owns only how it is written down.**

```
onnx_wire.pbi          bounded protobuf reader: varints, keys, spans, limits
onnx_model.pbi         model, graph, node, attribute, shape, tensor parsing
onnx_ir.pbi            target-neutral graph, constant folding, aliases,
                       lifetimes, arena planning
onnx_quant.pbi         eligible INT8 weight conversion, scales, layouts
onnx_storage.pbi       FP16/BF16/INT4 storage, rounding, decode budgets
onnx_pack.pbi          deterministic checked weight pack, identity, CRC
onnx_targets.pbi       the five profiles: dialect, kernels, capacity
onnx_compile.pbi       coordinator: options, validation, routing, manifests
onnx_emit.pbi          fixed-shape emission, the PmOnnx... API
onnx_forms.pbi         attribute forms both emitters accept or refuse,
                       and the refusal sentence
onnx_dynamic_emit.pbi  runtime-dimension emission, the PmModel... API,
                       and the runtime export
```

> **Trap.** Graph semantics must never branch on the target. If you find

yourself writing "on the Pico this operator means..." you are in the wrong file. A target selects a dialect, a kernel and a budget. The moment it changes what an operation computes, two targets have silently become two compilers and there is no reference left to check either against.

Which file should I change

```
Add a target              onnx_targets.pbi, then runtime export and launch
                          handling in both emitters
Add an operator           parser/IR validation, BOTH applicable emitters,
                          and BOTH relevant execution runtimes
Change a precision        onnx_quant.pbi or onnx_storage.pbi, then packing,
                          the runtime kernel or decoder, and capacity
Windows tensor speed      tensor_simd_windows.pbi
64-bit Arm tensor speed   tensor_fp32_neon.pmi
```

```
The generator window       onnx_ui.pbi
Pronunciation behaviour    kokoro_g2p.pmi, kokoro_g2p_extra.pmi, and the
                           adapters' normalization boundaries
```

"Both emitters" and "both runtimes" are not thoroughness for its own sake. A model with a symbolic extent and the same model with every extent pinned take different paths through this tree, and an operator added to one of them is an operator that works until somebody fixes their shapes.

The complete file-by-file guide is the source map reference page, and it covers every published file, including the ones this chapter has not.

The numerical contract

This is the part with the least room for judgement, so here it is as rules.

No fused multiply-add. Not in a kernel that has a scalar reference. An FMLA rounds once where the reference rounds twice, and that is a different number. It is usually a BETTER number, and it is still a different one, and "better" is not the contract.

No reassociation. Floating-point addition is not associative. Splitting a reduction into four partial accumulators and folding them at the end changes the answer. So the reductions in the 64-bit Arm runtime are deliberately scalar - that is not a kernel nobody got round to vectorizing, it is a kernel that must not be:

```
; Final-axis reductions keep the portable kernel's exact FP32 evaluation
; order while owning their hot loop state in registers.  In particular,
; these are deliberately scalar reductions: a horizontal vector reduction
; or several partial accumulators would reassociate non-associative adds.
```

No wider intermediate type. Evaluating a float32 expression in double and rounding at the end is the same class of change as fusing.

One floating-point control register, and it is left alone. The contract assumes nearest-even, gradual underflow and default-NaN disabled. That is what makes signed zeros, infinities, subnormals and NaN payloads come out of a vector kernel the same as they do one element at a time.

Say which kind of kernel you wrote. If it is bit-identical, claim it in the header and prove it. If it cannot be - a vector reduction, a fused path - say that in the header too, and make sure the gate keeps exact scalar checks alongside.

What the caller must have established

A fast kernel gets to skip checks precisely because something above it already did them. That contract belongs in the kernel's own header, in full, because it is the only place a reader will look:

```
; WHAT THE CALLER MUST HAVE ESTABLISHED, because this kernel checks none of
; it - DBinary does, and refuses everything else to the portable path:
;   *a, *b, *destination  FLOAT32, and *destination contiguous
;   aStride, bStride      1 for a contiguous operand, 0 for one broadcast
;                         element; nothing else
;   op                    0 add, 1 subtract, 2 multiply, 3 divide
;   destination           may be exactly *a or exactly *b when that operand
;                         is contiguous; it may NOT alias a BROADCAST
;                         operand ... and may not partially overlap either.
```

> **Trap.** Note the aliasing clause and why it exists: the scalar body re-reads

the broadcast element after each store and the vector body does not. Two implementations of "the same" operation differ in a case the caller can reach. That is exactly the kind of fact that is obvious for a week and invisible forever afterwards, and the only defence is writing it down where the next person is already looking.

How a kernel is proved

Roughly in this order. Skipping steps is how a fast wrong kernel gets shipped.

1. State the contract first. Bit-identical to the reference, or a tolerance with a stated instrument. Write it in the header before writing the kernel, because it determines the kernel's shape.

2. Differential test against the reference it replaces. Real inputs, plus the awkward ones: zeros of both signs, infinities, NaNs and their payloads, subnormals, the reduction boundaries, every alignment, the one-two-three-element tails, the broadcast cases, the aliasing cases the header permits.

3. Negative controls. Damage the kernel on purpose and confirm the gate fails. A gate that has never gone red has not been shown to be capable of it. Name the mutants; a suite that catches eleven planted defects is a suite you can quote.

4. Run it where it will actually run. A kernel that passes in an instruction interpreter and has never executed on the silicon is a kernel with one unproven step left, and it is the step that has surprised people before. Where hardware exists, the gate runs on the hardware.

5. Prove it at model scale. The fixture says the kernel is right. The model says the kernel is right *in this program*. For the speech model that means the output waveform's digest, unchanged.

6. Keep the controls. When you measure a speedup, leave something untouched in the same measurement. Two operator families coming back identical to the millisecond is what turns "it got faster" into "these kernels got faster".

> **Trap.** Put the gate somewhere it is tracked. `tests/` in this repository is

ignored apart from `tests/node_suite/`, so a gate written anywhere else there is on one disk and nowhere else - see chapter 8. An operator added or fixed should move the node-test score, and the score before and after belongs with the change. This is not a filing preference. A proof that does not outlive a working directory is not a proof.

Licensing and notices

The project source and documentation here are **MIT**: use, modify and redistribute, including commercially, keeping the licence notice. The software is provided as is, with the licence's limitation of liability.

That licence covers what is in this repository. It does not relicense anything you obtain separately, and several things in this book are obtained separately:

```
the host language   required to build the tools and Windows output
PureMetal Forge     separate toolchain for the bare-metal dialects
ONNX Runtime        optional host reference/trace engine only
Kokoro              optional speech model and voices
Misaki / CMUdict    source data for the pronunciation packs
```

Trap. A prepared pack is derived data. Turning a dictionary into a

`.pmg2p` or a raw voice into a `.pmvoice` changes the format, not the licence. Keep the notices for the **specific versions** you downloaded, transformed or redistributed. This compiler grants you no rights to model assets it does not contain.

No proprietary compiler binaries, private keys, model data or signing material are in this repository, and none should be added to it.

Before you send a change

Rebuild the generator if you touched anything under `runtime/` - the generator embeds it, so an unrebuilt tool cannot emit your change. Regenerate the demo closures and check that what changed is what you meant to change. Run `--self-test`. Run the arithmetic walkthrough end to end.

And if your change alters what a developer DOES - a command, a flag, a file format, an emitted API, a capacity rule, an install step - update the chapter of this book that describes it and re-render the PDF in the same commit. A book that lags its repository is a defect like any other, and the rendering command is one line. It is in chapter 10.

Where everything is

The reference pages, what each one answers, and how this book is rendered and kept honest.

This book and the reference pages are different things

The book is a path through the product: read it in order and you end up able to use the thing. The reference pages under `docs/` are lookups: go straight to the one that owns your question and read only that.

Where they overlap, they are meant to agree. If they ever disagree, one of them is wrong and it is worth saying so rather than working around it.

The map

```
README.md              What the project is, in one screen, with the badges
                       and the links. Start here if you arrived by
                       accident.

docs/QUICKSTART.md     The arithmetic walkthrough as a checklist: build,
                       create the demo models, generate, run. Chapter 3 is
                       the same ground with the reasoning attached.

docs/BUILD.md          Building the two tools. Requirements, the helper,
                       the manual commands, why runtime/ is needed at
                       build time, and the optional private release hook.
                       The authority for chapter 2.

docs/SOURCE_MAP.md     EVERY published source file and what it is
                       responsible for, plus a "which file should I
                       change?" table. The authority for chapter 9's
                       opening, and considerably more complete than it.

docs/ARCHITECTURE.md   The pipeline as a diagram, the boundary where this
                       tool stops, graph and memory planning, the two
                       emission paths, why the source closure works the
                       way it does.

docs/TARGETS.md        Target-by-target integration: what you build each
                       one with, what your application must supply, the
                       two emitted APIs step by step, the precision
                       table, Pico capacity, and the 64-bit Arm compiler
                       compatibility requirements. The authority for
                       chapters 5 and 6.

docs/KOKORO.md         Speech: what is included, the four assets and their
                       exact contracts, the pack and verify commands, the
                       Windows reader, the portable adapter, and the
                       measured Pi 4 status. The authority for chapter 7.

docs/VALIDATION.md     What was checked, when, and what each check
                       establishes - plus the explicit limitations, in a
                       list. The authority for chapter 8.
```

```
docs/ResidentProgress.md
                      The progress-callback contract in full: the four
                      phases, cancellation, the non-reentry rule, and
                      what progress points do and do not bound.

examples/pi4/         The Raspberry Pi 4 examples. kokoro-sentence/ is
                      the runner, the two host tools and the recipe
                      behind chapter 11, and its README is the
                      authority for that recipe.

tests/node_suite/     The node-test coverage harness, its generic driver,
                      the targeted default-attribute and small-kernel
                      cases, and the
                      published scores in results/. Developer checks;
                      chapter 8 says how to run them.

THIRD_PARTY_NOTICES.md Everything obtained separately, what its
                      relationship to this project is, and where to check
                      its terms.

LICENSE               MIT.
```

Two things that are deliberately not here

The gates. `tests/` is ignored by this repository apart from the node-test harness in `tests/node_suite/`, and the internal diagnostics are not part of this export. Chapter 8 says where they live and why that is the rule rather than an omission.

The speech assets. No model, no voices, no dictionary payloads. Chapter 7 and the speech reference page both say what the contracts are, so you can obtain or prepare compatible ones.

How this book is rendered

The chapters are the `.txt` files beside this one, in filename order. Files beginning `00_` are documentation of the book and are never rendered; `00_OUTLINE.txt` explains the markup and the rules the chapters are written to.

They are printed by the same renderer that prints the PureMetal Forge books, pointed at this folder, so this book looks like those books and cannot drift into its own private dialect of the markup:

```
python tools/build_help_pdf.py ^
    --book <this checkout>\docs\guide ^
    --out  <this checkout>\docs\guide\OnnxCompilerGuide.pdf ^
    --title "The ONNX Compiler Guide" ^
    --subtitle "Ahead-of-time ONNX compilation to readable BASIC source." ^
    --section-nav
```

It writes exactly one PDF exactly where you said, and nothing else. The `--section-nav` flag puts the section headings into the contents as well as the chapter titles, which on a reference book of this size is worth the extra page.

> **Note.** The renderer is part of the PureMetal Forge tooling, not of this

repository, because it is one renderer serving several books. A second renderer maintained here would quietly drift from the first, and the two books would slowly stop looking like one product.

Keeping it true

One rule governs this book, and it is worth stating where a contributor will meet it:

A commit that changes what a developer does updates the affected chapter and re-renders the PDF in the same commit. A command, a flag, a file format, an emitted API, an install step, a capacity rule, a board procedure. Same commit, not a follow-up.

The reason is the same reason chapter 4 is full of exact refusal text. A reader who copies a command from documentation and gets an error does not conclude that this one line is stale. They conclude that the documentation lies, and they stop trusting all of it - including the parts that would have saved them a day. A book that lags its repository is a defect, and it gets filed like one.

Two habits keep it cheap. **Print only what you ran**: every command in this book was executed, or copied verbatim from `--help`, and every refusal quoted was produced by making that mistake on purpose. And **name pending work as pending, in the same sentence as the thing it will change**, so a reader can tell the product from the plan without having to guess which they are reading.

Running a model on the Raspberry Pi 4: the Kokoro sentence

One sentence of speech from the full model, on a board with no operating system, on four cores - and every step from a clean checkout to the WAV.

What this chapter reproduces

On 2026-09-16 a Raspberry Pi 4 running the Anvil monitor, with nothing underneath it, ran the full Kokoro-82M model of chapter 7 in FP32 on all four cores and spoke this sentence:

```
I am the kokoro speech AI model on the raspberry Pi 4. How do I sound,
good? I am running on bare metal, with no operating system, under Anvil,
a mini kernel and boot loader, on all four cores of the Raspberry Pi 4.
Both Anvil and I were compiled by Pure Metal Forge.
```

Everything that made it is published in `examples/pi4/kokoro-sentence/`: the runner that was entered on the board, two host tools, and a README that is the step-by-step recipe with every expected hash. This chapter is the same path with the reasoning attached. The README is the one to follow with a board in front of you.

```
examples/pi4/
  README.md                   the Pi 4 examples, one line each
  kokoro-sentence/
    README.md                 the recipe
    KokoroSentence.pi4        the runner - an Anvil payload
    node_output_map.pbi       a table the runner's optional NaN scan reads
    kokoro_readback.py        host tool: trace, result, dispatch count, waveform
    f32_to_wav.py             host tool: raw samples to WAV
```

> **Note.** The two `.py` files are developer tools for talking to a board.

Building this compiler, and generating or building a model with it, needs no Python. The tools use the Anvil repository's own host code by path - its console client in `tools/board_run.py` and its verified reader in `tools/anvil_readback.py` - rather than a copy of either.

What it measured

```
board                Raspberry Pi 4 Model B, 4 GB, Anvil build 142
tokens               295
samples              453,000 FP32 at 24 kHz, mono: 18.875 s of audio
time                 158.2 s from the start of the model bind to the end
                     of the request (8,540,801,559 ticks at 54 MHz),
                     about 8.4 times slower than real time
four-core dispatches 84, read from the monitor's counter after the run on
                     a monitor reset just before it
payload status       0; trace stage 9 at node 2462
raw samples          1,812,000 B  sha256 05c30419a882ad6a094a05fe0b7bbe4e
                     96d549c20896a5522991c652d5296f6f, equal to the
                     board's own sha256sum of the range
```

```
WAV, 16-bit PCM       906,044 B  sha256 a764205813ee082c365ccd510cb24402
                     f6357e452c3cfb584556295513207080, no sample clamped
```

Against ONNX Runtime 1.29.0 on a PC, with the same text, the same two pronunciation packs and the same voice, every one of the 453,000 samples compared with no alignment and no trimming:

```
max absolute difference   0.259917
RMSE                      0.006588
correlation               0.996006
```

> **Trap.** 0.26 is above the 0.1 maximum absolute difference set for the short

test sentences, and it is recorded as a failure of that limit. The board's kernels are held bit-identical to the scalar reference, and the four-core split was proven to return the same bytes as one core on the shorter sentences, so the difference lies between the reference and ONNX Runtime. Chapter 8 traces the mechanism on a shorter sentence - a tiny divergence in the text encoder amplified into the phase of a sine with very large arguments. This sentence is 4.4 times longer and diverges further, which fits that mechanism; it has not been re-traced node by node.

What you need

A Raspberry Pi 4 Model B with at least 4 GB: the runner's working memory ends at `$A0000000`. A FAT32 USB stick or SD card for Anvil; the run booted from a USB stick that also held the weight pack. An Ethernet cable straight to a Windows PC. On the PC: Git, PureBasic 6.41 x64 (its C back end) for this compiler, PureMetal Forge from puremetalforge.ajtaji.com for the payload, and Python 3 for the host tools, with pyserial for Anvil's upload tool. The Forge download also includes a ready-built copy of this compiler. The 2026-09-14 copy cannot build this example; the 2026-09-17 copy can, but not the exact payload that ran. The section after the two builds shows both.

The inputs, pinned

The model and the voice come from the Kokoro-82M v1.0 ONNX repository at revision `1939ad2a...`; the dictionaries from Misaki at `fba12365...` and CMUdict at `74790861...`:

```
> $hf = 'https://huggingface.co/onnx-community/Kokoro-82M-v1.0-
ONNX/resolve/1939ad2a8e416c0acfeecc08a694d14ef25f2231'
> curl.exe -L -o downloads\model.onnx "$hf/onnx/model.onnx"
> curl.exe -L -o downloads\af_heart.bin "$hf/voices/af_heart.bin"
> git clone -c core.autocrlf=true https://github.com/hexgrad/misaki.git
> git -C misaki checkout fba1236595f2d2bf21d414ba6e57d25256afada3
> git clone -c core.autocrlf=false https://github.com/cmusphinx/cmudict.git
> git -C cmudict checkout 74790861f652b15e4ac49015a90074ad62a27690

model.onnx       325,532,232 B  8fbea51ea711f2af382e88c833d9e288c6dc82ce5e98421ea61c058ce21a34cb
af_heart.bin         522,240 B  d583ccff3cdca2f7fae535cb998ac07e9fcb90f09737b9a41fa2734ec44a8f0b
us_gold.json       3,093,057 B  a13911134e702c8fd1d79ea47cdc64f146aec1deabf4638a9a8cdd71da5b87a3
us_silver.json     3,192,879 B  a7d8629ff02614cd37837f44c41f0085f55bed4f0eb2c169964f0077bb102d6b
cmudict.dict       3,618,488 B  81917843c7f44ce2b094ac63873c2c7a4cf802040792c455ba3ca406891c3d22
```

Trap. The line endings are part of the pin. The Misaki digests are of

the files with CRLF line endings, which is what a clone with `core.autocrlf=true` produces; the same files downloaded raw have LF endings, a different size and a different digest, and the pack builder refuses them before parsing a byte. The CMUdict digest is the other way round, which is why its clone turns `core.autocrlf` off. Both clones were run and give the digests above.

Note. The two files from the model repository were not downloaded again

when this recipe was followed on the desk: copies already on disk with the same SHA-256 were used, and the two URLs were checked against the size and SHA-256 the server publishes for each file.

Trap. Keep the working folder's path short. The model's runtime files are

written several folders deep inside it, and a path past the classic 260-character Windows limit makes the compile fail with "Cannot create generated runtime source", a sentence that does not mention the length (forum topic 868, open). `$HOME\pi4-kokoro` is short enough.

Two builds of the compiler, and why

The model source that ran was emitted by this compiler at commit `6b6b770`. Commits after it changed runtime kernels, so they emit a different payload - a payload built from `main` has not been run on a board. The supplementary pronunciation pack needs `main`, because until `b48cd05` the native packer could not build the pack its own verifier requires. So the recipe builds both, the older one under a second name:

```
> & $pb src\PureMetalOnnxCompiler.pb /CONSOLE /THREAD /OPTIMIZER /OUTPUT bin\PureMetalOnnxCompilerCLI.exe
> git checkout 6b6b770
> & $pb src\PureMetalOnnxCompiler.pb /CONSOLE /THREAD /OPTIMIZER /OUTPUT bin\OnnxCli-6b6b770.exe
> git checkout main
```

`bin\` is ignored by Git, so both executables survive the checkouts.

The compiler in the PureMetal Forge download

The PureMetal Forge download includes this compiler ready to run, in `PureMetalForge\tools\onnx\`, and chapter 2 recommends it. Which release you have decides what it can do here.

The 2026-09-14 release's copy cannot build this example. It was built on 2026-09-13, before the model source gained `PmOnnxAnvilEnter`. On 2026-09-16 the model and pack steps were run with it from fresh clones, in place of the two builds above:

```
> $onnx = "$root\PureMetalForge\tools\onnx\PureMetalOnnxCompilerCLI.exe"
> & $onnx --compile "$root\downloads\model.onnx" --output "$ex\model\kokoro-pi4" --target pi4 --precision fp32
--kokoro-text

kokoro-pi4.pmw          324,621,440 B  a4117721...   the same
```

```
af_heart.pmvoice            522,368 B  79ef729c...   the same
kokoro_us_english.pmg2p  12,613,954 B  0d791ecc...   the same
kokoro_us_extra.pmg2p     5,397,230 B  e9eee914...   older: 66,304 entries
kokoro-pi4.pi4              788,990 B  accb8c92...   older: no PmOnnxAnvilEnter
```

The supplementary pack is the one from before forum topic 871, and a compiler built from `main` refuses it with `PMG2P-3020`. The model source has older kernels and no four-core entry, so the payload build stops:

```
SEMANTIC ERROR: 'PmOnnxAnvilEnter' at KokoroSentence.pi4 line 397, in procedure krbindservices() is not
defined - there is no procedure and no intrinsic with that name.
pmfc: BUILD FAILED - no output produced.
```

The 2026-09-17 release's copy is commit `658566d`. Run the same way on 2026-09-17, it gave the weight pack and all three packs byte-identical to the tables below, the 5,397,245-byte supplement included, and the runner built with it. The payload was different, because the model source comes from later runtime kernels (789,716 bytes, not 789,585):

```
build\kokoro-sentence.img.pmf   1,590,676 B  61668df99f5c7d7d34f42f3409b97872bd424662cd4964ac776ed9a250a7285f
```

That payload has not been run on a board. To reproduce the exact payload that ran, build the compiler from source as this chapter does.

The model and the assets

The model is emitted beside the runner, because the runner includes it by a relative path:

```
> .\bin\OnnxCli-6b6b770.exe --compile "$root\downloads\model.onnx" --output "$ex\model\kokoro-pi4" --target
pi4 --precision fp32 --kokoro-text

kokoro-pi4.pmw          324,621,440 B  a41177210505c80c08836acbb919283d140cb94ddfce89ed2dc732ed8c0d5413
kokoro-pi4.pi4              789,585 B  1dc51126a2ef266a84a9694d47358c1f51509513c3efd6b32a9a9b58e5b5eafe
kokoro-pi4.kokoro.pi4           243 B  b443f672ea10401e4bc35d87192f8ee6f4a1744b688d803e8e6025ebd2dbb397
```

The weight pack is the file that ran. The source differs from the run's in exactly one way: the output name appears in five include lines, and the run used another name. The runtime folder's nine files match the run's once line endings are normalised, because a checkout gives them CRLF. Neither difference reaches the compiled payload.

```
> .\bin\PureMetalOnnxCompilerCLI.exe --kokoro-pack-voice "$root\downloads\af_heart.bin" --output
"$root\assets\af_heart.pmvoice" --source-sha256
d583ccff3cdca2f7fae535cb998ac07e9fcb90f09737b9a41fa2734ec44a8f0b
> .\bin\PureMetalOnnxCompilerCLI.exe --kokoro-pack-base-dictionary "$root\misaki\misaki\data\us_gold.json" --
silver "$root\misaki\misaki\data\us_silver.json" --output "$root\assets\kokoro_us_english.pmg2p"
> .\bin\PureMetalOnnxCompilerCLI.exe --kokoro-pack-extra-dictionary "$root\cmudict\cmudict.dict" --base
"$root\assets\kokoro_us_english.pmg2p" --output "$root\assets\kokoro_us_extra.pmg2p"

kokoro_us_english.pmg2p   12,613,954 B  0d791ecc12e11d77b5d961718128cf945a67ab53274816a94b9d8eecddbc0b88  ->
$54000000
kokoro_us_extra.pmg2p      5,397,245 B  73be678aa7d1e8cd1b8f87681f4ee1a14f26ac7298fad1063136813064ea8525  ->
$55000000
af_heart.pmvoice             522,368 B  79ef729cc9aa7ec47025ece5c3d6fa230404e0f4d061465d47a71afc4b3d06ce  ->
$56000000
```

All three are byte-identical to the assets the board bound.

The runner

`KokoroSentence.pi4` is chapter 7's resident runner in outline, made real. Its comments walk through every step; the shape is:

```
Main():
  KrCaptureX0()              keep x0, the service table - first statement
  zero the tick rows and the KTRC trace
  KrBindServices()           check the table's magic, version and slot count,
                             take the deadman slot, and
    PmOnnxAnvilEnter(table)  hand the table to the model: four cores
  MmuBuildTables, MmuEnableCached    the payload's own page tables and caches
  PmModelBindMemory($40000000, 324621440, $60000000, 1 GiB)
  PmKokoroTextBind($54000000, ..., $55000000, ..., $56000000, ...)
  KrRender(text, $57400000)  text -> tokens -> 453,000 samples; hash them,
                             copy them out of the borrowed output tensor
  KrFinish(0)                release everything, check nothing leaked,
                             turn the MMU off, write trace stage 9, return
```

Around that it keeps three records a host can read after the run: the KTRC trace at `$57E00000`, rewritten at every node, which says how far the run got even when the console has lost the return line; one row of per-node timer ticks per request from `$57D00000`; and a block of result globals whose addresses are in the `.sym` file the compiler writes beside the image. Its return value, `x0`, is a status: 0, or a code the file's `KrFinish` comment lists.

> **Note.** The file still carries the result fields of two earlier requests,

their texts, and a per-node NaN scan that is switched off. They are compiled in, unused, so that this source builds the exact payload that ran; removing them is safe and changes the payload's hash.

The payload, byte for byte

```
> $build = Start-Process $pmf -ArgumentList '--compile KokoroSentence.pi4 -t pi4 --load-addr 0x500000 --bss-
addr 0x1000000 --stack-addr 0x7F00000 --entry-returns --wants-services -o build\kokoro-sentence.img' -Wait -
PassThru -NoNewWindow

build\kokoro-sentence.img.pmf   1,578,488 B  58b402ea88f92c414a752df39f5ef4b006021a905018f3145df3fb4b02da63b3
```

That is the container that was entered on the board on 2026-09-16. It was reproduced from fresh clones of this repository and Anvil with the PureMetal Forge download of 2026-09-14 (`PureMetalForge.exe` sha256 `8f2e43d4...`). The 2026-09-17 release's `PureMetalForge.exe` (sha256 `5898e744...`) builds the same bytes from the same `6b6b770` model source.

On the board

Anvil's boot-volume skeleton, `Boards/RaspberryPi4/sdcard` in the Anvil repository, carries a `kernel8.img` and the rest of the card. The steps from there are uploads, one run and a readback, and one command does all of them from the manifest `examples/pi4/kokoro-sentence.run.json` :

```
> $tools = "$root\Anvil\tools"
> python "$tools\board_model_run.py" "$ex\..\kokoro-sentence.run.json" --console-ip $board --no-lease
```

Anvil's `board_model_run.py` asks the board for the `sha256sum` of each asset's range first and uploads only an asset whose digest differs, then uploads the payload, arms the 15-second deadman, enters it, and the moment it returns reads the 1,812,000 samples back, checks them against the board's own `sha256sum` and prints `WAV ready:` with the path of `run\<time>\kokoro-sentence.wav`. On a shared board, `--lease-file` and `--lane` replace `--no-lease` and it takes and releases the lease itself.

> **Note.** The tool was proven on a board on 2026-09-16 with the Kitten TTS

manifest beside this one (`kitten-rosie.run.json`); this Kokoro manifest has not yet been run. The steps it replaces are below, as the run used them.

These are the commands the run used, with this recipe's paths:

```
> python "$tools\pi4_upload.py" "$root\assets\kokoro_us_english.pmg2p" --console net --console-ip $board --
board-ip $board --addr 0x54000000 --cache
  VERIFIED: 12613954 bytes, sha256 equal, 1389 ms on the board (8868 KB/s).
  (and the same for the supplement at 0x55000000 and the voice at 0x56000000)

pmf> load MODEL.PMW 40000000
Loaded 324621440 bytes into memory, filling 40000000 through 5359547F.

> python "$tools\board_run.py" "$ex\build\kokoro-sentence.img.pmf" --console-ip $board --board-ip $board --
addr 0x03000000 --deadman 15 --expect-x0 0 --timeout 700 --out "$ex\run" --name kokoro-sentence
  stage   03000000
  sent 1578488 bytes in 0.17 s
  verified on the board: 1578488 bytes, digest matches
  deadman 15 s
  boot mem 3000000 shot

pmf> sha256sum 57400000 1BA620
sha256sum over 1812000 bytes, 57400000 .. 575BA61F ...
The sha256sum is 05c30419a882ad6a094a05fe0b7bbe4e96d549c20896a5522991c652d5296f6f
```

The weights did not go over the wire on the day: they had been uploaded and verified before and saved to the USB stick as `MODEL.PMW`, and after the reset before the run the board loaded them back. Uploading them with `pi4_upload.py` to `0x40000000` works the same way. Either way the payload checks the pack's header, identity and CRC-32 when it binds.

> **Trap.** ** `board_run.py` said the payload never returned, and it had

finished.** The console lost the return line during a long silent run. Do not run it again: read the trace. That is what the trace is for, and what `kokoro_readback.py status` reads.

```
> python "$ex\kokoro_readback.py" cores --console-ip $board       before and after the run: the difference is
the dispatch count
> python "$ex\kokoro_readback.py" status --console-ip $board      trace, status, tokens, samples
```

```
> python "$ex\kokoro_readback.py" wave --console-ip $board       board sha256sum, readback, sha256sum again
> python "$ex\f32_to_wav.py"                                    run\kokoro-sentence.wav
```

`wave` saves the samples only if the board's digest before the read, the board's digest after it and the host's digest of what arrived all agree, and the XOR of the samples equals the one the payload computed on the board before it copied them.

Trap. Two parts of this section are not yet a board transcript.

`kokoro_readback.py` passes a self-test against a scripted console and has not been run against a board; on the day, the waveform came back through build 142's older `memory` dump path, which took 1,048 s. It uses the `readback` command of Anvil build 151, which the Anvil repository measured reading the same 1,812,000 bytes in 1.28 s. `f32_to_wav.py`, run on the day's samples afterwards on the desk, writes the published WAV byte for byte.

Speaking something else

Change the bytes under `KrText3` - one `Data.a` value per UTF-8 byte - and `#KR_TEXT3_BYTES`. PowerShell produces both:

```
> [Text.Encoding]::UTF8.GetBytes('Hello from the Pi.') -join ','
72,101,108,108,111,32,102,114,111,109,32,116,104,101,32,80,105,46
> [Text.Encoding]::UTF8.GetByteCount('Hello from the Pi.')
18
```

Set `#KR_EXPECT_TOKENS3` to the sentence's token count if you know it. If it is wrong the samples are still rendered and copied, the payload returns 320 instead of 0, and `status` prints the real count. The copy slot holds 524,288 samples, 21.8 seconds; a longer sentence is refused with 313, not cut short. A changed runner is a different payload with a different hash.